beginning

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

data loading

```
agg_encoded_20 = pd.read_excel("/content/drive/MyDrive/capstone/datasets/agg/agg_df_encoded_20.xlsx")
agg_encoded_40 = pd.read_excel("/content/drive/MyDrive/capstone/datasets/agg/agg_df_encoded_40.xlsx")

agg_knn_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/agg/agg_df_knn_context.xlsx")
agg_knn_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/agg/agg_df_knn_no_context.xlsx")

agg_mice_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/agg/agg_df_mice_context.xlsx")
agg_mice_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/agg/agg_df_mice_no_context.xlsx")

agg_rf_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/agg/agg_df_rf_context.xlsx")
agg_rf_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/agg/agg_df_rf_no_context.xlsx")
```

```
dbscan_encoded_20 = pd.read_excel("/content/drive/MyDrive/capstone/datasets/dbscan/dbscan_df_encoded_20.xlsx")
dbscan_encoded_40 = pd.read_excel("/content/drive/MyDrive/capstone/datasets/dbscan/dbscan_df_encoded_40.xlsx")

dbscan_knn_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/dbscan/dbscan_df_knn_context.xlsx")
dbscan_knn_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/dbscan/dbscan_df_knn_no_context.xlsx")

dbscan_mice_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/dbscan/dbscan_df_mice_context.xlsx")
dbscan_mice_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/dbscan/dbscan_df_mice_no_context.xlsx")

dbscan_rf_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/dbscan/dbscan_df_rf_context.xlsx")
dbscan_rf_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/dbscan/dbscan_df_rf_no_context.xlsx")
```

```
kmeans_encoded_20 = pd.read_excel("/content/drive/MyDrive/capstone/datasets/kmeans/kmeans_df_encoded_20.xlsx")
kmeans_encoded_40 = pd.read_excel("/content/drive/MyDrive/capstone/datasets/kmeans/kmeans_df_encoded_40.xlsx")

kmeans_knn_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/kmeans/kmeans_df_knn_context.xlsx")
kmeans_knn_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/kmeans/kmeans_df_knn_no_context.xlsx")

kmeans_mice_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/kmeans/kmeans_df_mice_context.xlsx")
kmeans_mice_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/kmeans/kmeans_df_mice_no_context.xlsx")

kmeans_rf_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/kmeans/kmeans_df_rf_context.xlsx")
kmeans_rf_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/kmeans/kmeans_df_rf_no_context.xlsx")



spc_encoded_20 = pd.read_excel("/content/drive/MyDrive/capstone/datasets/spc/spc_df_encoded_20.xlsx")
spc_encoded_40 = pd.read_excel("/content/drive/MyDrive/capstone/datasets/spc/spc_df_encoded_20.xlsx")

spc_knn_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/spc/spc_df_knn_context.xlsx")
spc_knn_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/spc/spc_df_knn_no_context.xlsx")

spc_mice_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/spc/spc_df_mice_context.xlsx")
spc_mice_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/spc/spc_df_mice_no_context.xlsx")

spc_rf_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/spc/spc_df_rf_context.xlsx")
spc_rf_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/spc/spc_df_rf_no_context.xlsx")
```

dataframe grouping

```
agg_dfs = [
    # Agglomerative
    (agg_encoded_20, "agg_encoded_20"),
    (agg_encoded_40, "agg_encoded_40"),
    (agg_knn_context, "agg_knn_context"),
    (agg_knn_no_context, "agg_knn_no_context"),
    (agg_mice_context, "agg_mice_context"),
    (agg_mice_no_context, "agg_mice_no_context"),
    (agg_rf_context, "agg_rf_context"),
    (agg_rf_no_context, "agg_rf_no_context"),
]

dbscan_dfs = [
    # DBSCAN
    (dbscan_encoded_20, "dbscan_encoded_20"),
    (dbscan_encoded_40, "dbscan_encoded_40"),
    (dbscan_knn_context, "dbscan_knn_context"),
    (dbscan_knn_no_context, "dbscan_knn_no_context"),
    (dbscan_mice_context, "dbscan_mice_context"),
    (dbscan_mice_no_context, "dbscan_mice_no_context"),
    (dbscan_rf_context, "dbscan_rf_context"),
    (dbscan_rf_no_context, "dbscan_rf_no_context"),
]

kmeans dfs = [
```

```
    # KMeans
    (kmeans_encoded_20, "kmeans_encoded_20"),
    (kmeans_encoded_40, "kmeans_encoded_40"),
    (kmeans_knn_context, "kmeans_knn_context"),
    (kmeans_knn_no_context, "kmeans_knn_no_context"),
    (kmeans_mice_context, "kmeans_mice_context"),
    (kmeans_mice_no_context, "kmeans_mice_no_context"),
    (kmeans_rf_context, "kmeans_rf_context"),
    (kmeans_rf_no_context, "kmeans_rf_no_context")
]

spc_dfs = [
    # spc
    (spc_encoded_20, "spc_encoded_20"),
    (spc_encoded_40, "spc_encoded_40"),
    (spc_knn_context, "spc_knn_context"),
    (spc_knn_no_context, "spc_knn_no_context"),
    (spc_mice_context, "spc_mice_context"),
    (spc_mice_no_context, "spc_mice_no_context"),
    (spc_rf_context, "spc_rf_context"),
    (spc_rf_no_context, "spc_rf_no_context")
]


# Combine all three into one list
all_dfs = agg_dfs + dbscan_dfs + kmeans_dfs + spc_dfs


# extra groups for cross comparison
no_context_dfs = [
    (agg_knn_no_context, "agg_knn_no_context"),
    (agg_mice_no_context, "agg_mice_no_context"),
    (agg_rf_no_context, "agg_rf_no_context"),
    (spc_knn_no_context, "spc_knn_no_context"),
    (dbscan_knn_no_context, "dbscan_knn_no_context"),
    (dbscan_mice_no_context, "dbscan_mice_no_context"),
    (spc_mice_no_context, "spc_mice_no_context"),
    (dbscan_rf_no_context, "dbscan_rf_no_context"),
    (kmeans_knn_no_context, "kmeans_knn_no_context"),
    (kmeans_mice_no_context, "kmeans_mice_no_context"),
    (kmeans_rf_no_context, "kmeans_rf_no_context"),
    (spc_rf_no_context, "spc_rf_no_context"),
]

context_dfs = [
    (agg_knn_context, "agg_knn_context"),
    (agg_mice_context, "agg_mice_context"),
    (agg_rf_context, "agg_rf_context"),
    (dbscan_knn_context, "dbscan_knn_context"),
    (dbscan_mice_context, "dbscan_mice_context"),
    (dbscan_rf_context, "dbscan_rf_context"),
    (kmeans_knn_context, "kmeans_knn_context"),
    (kmeans_mice_context, "kmeans_mice_context"),
    (kmeans_rf_context, "kmeans_rf_context"),
    (spc_knn_context, "spc_knn_context"),
    (spc_mice_context, "spc_mice_context"),
    (spc_rf_context, "spc_rf_context")
```

```
    (spc_rf_context, "spc_rf_context"),
]

no_imputed_dfs = [
    (agg_encoded_20, "agg_encoded_20"),
    (agg_encoded_40, "agg_encoded_40"),
    (dbscan_encoded_20, "dbscan_encoded_20"),
    (dbscan_encoded_40, "dbscan_encoded_40"),
    (kmeans_encoded_20, "kmeans_encoded_20"),
    (kmeans_encoded_40, "kmeans_encoded_40"),
    (spc_encoded_20, "spc_encoded_20"),
    (spc_encoded_40, "spc_encoded_40"),
]

mice_dfs = [
    (agg_mice_context, "agg_mice_context"),
    (agg_mice_no_context, "agg_mice_no_context"),
    (dbscan_mice_context, "dbscan_mice_context"),
    (dbscan_mice_no_context, "dbscan_mice_no_context"),
    (kmeans_mice_context, "kmeans_mice_context"),
    (kmeans_mice_no_context, "kmeans_mice_no_context"),
    (spc_mice_context, "spc_mice_context"),
    (spc_mice_no_context, "spc_mice_no_context"),
]

rf_dfs = [
    (agg_rf_context, "agg_rf_context"),
    (agg_rf_no_context, "agg_rf_no_context"),
    (dbscan_rf_context, "dbscan_rf_context"),
    (dbscan_rf_no_context, "dbscan_rf_no_context"),
    (kmeans_rf_context, "kmeans_rf_context"),
    (kmeans_rf_no_context, "kmeans_rf_no_context"),
    (spc_rf_context, "spc_rf_context"),
    (spc_rf_no_context, "spc_rf_no_context")
]

knn_dfs = [
    (agg_knn_context, "agg_knn_context"),
    (agg_knn_no_context, "agg_knn_no_context"),
    (dbscan_knn_context, "dbscan_knn_context"),
    (dbscan_knn_no_context, "dbscan_knn_no_context"),
    (kmeans_knn_context, "kmeans_knn_context"),
    (kmeans_knn_no_context, "kmeans_knn_no_context"),
    (spc_knn_context, "spc_knn_context"),
    (spc_knn_no_context, "spc_knn_no_context"),
]
```

general evaluation function

```
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score
from scipy.spatial.distance import cdist
import numpy as np

def evaluate_clusters(
```

```
        df,
        df_name,
        sil_metric="euclidean",  # match your clustering distance
        sample_size=None,
        random_state=42,
        return_dict=False
    ):
        """
        General-purpose clustering evaluation function.
        Works for Agglomerative, DBSCAN, KMeans, Spectral, GMM, etc.
        """

        rng = np.random.RandomState(random_state)

        label_col = df_name.split("_")[0] + "_cluster"

        X = df.drop(columns=[label_col]).values
        labels = df[label_col].values

        # Optionally ignore a label (e.g., DBSCAN noise = -1)
        if df_name.split("_")[0] == 'dbscan':
            mask = labels != -1
            X, labels = X[mask], labels[mask]

        # Optional subsample for speed (applies to all metrics)
        if sample_size is not None and X.shape[0] > sample_size:
            idx = rng.choice(X.shape[0], size=sample_size, replace=False)
            X, labels = X[idx], labels[idx]

        unique_clusters, counts = np.unique(labels, return_counts=True)
        n_clusters = len(unique_clusters)

        if n_clusters <= 1:
            print("⚠ Only one cluster found. Metrics cannot be computed robustly.")
            return {} if return_dict else None

        results = {}

        # Core indices
        try:
            results["Silhouette Score"] = {
                "score": silhouette_score(X, labels, metric=sil_metric),
                "range": "[-1, 1]",
                "better": "higher"
            }
        except Exception:
            results["Silhouette Score"] = {"score": np.nan, "range": "[-1, 1]", "better": "higher"}

        try:
            results["Calinski–Harabasz Index"] = {
                "score": calinski_harabasz_score(X, labels),
                "range": "[0, ∞)",
                "better": "higher"
            }
        except Exception:
            results["Calinski–Harabasz Index"] = {"score": np.nan, "range": "[0, ∞)", "better": "higher"}
```

```
    try:
        results["Davies–Bouldin Index"] = {
            "score": davies_bouldin_score(X, labels),
            "range": "[0, ∞)",
            "better": "lower"
        }
    except Exception:
        results["Davies–Bouldin Index"] = {"score": np.nan, "range": "[0, ∞)", "better": "lower"}

    # Dunn Index
    def dunn_index(X, labels):
        eps = 1e-12
        uniq = np.unique(labels)
        # Intra-cluster diameters
        intra = []
        for k in uniq:
            pts = X[labels == k]
            if pts.shape[0] >= 2:
                d = cdist(pts, pts)
                intra.append(np.max(d))
            else:
                intra.append(0.0)  # singleton cluster
        denom = max(max(intra), eps)

        # Inter-cluster minimum distance
        inter_min = np.inf
        for i in range(len(uniq)):
            for j in range(i + 1, len(uniq)):
                Xi = X[labels == uniq[i]]
                Xj = X[labels == uniq[j]]
                d = cdist(Xi, Xj)
                inter_min = min(inter_min, np.min(d))
        if not np.isfinite(inter_min):
            return np.nan
        return inter_min / denom

    # Xie–Beni Index
    def xie_beni_index(X, labels):
        eps = 1e-12
        uniq = np.unique(labels)
        centroids = np.array([X[labels == k].mean(axis=0) for k in uniq])
        # Compactness
        compact = 0.0
        for i, k in enumerate(uniq):
            pts = X[labels == k]
            diff = pts - centroids[i]
            compact += np.sum(diff * diff)
        # Min centroid separation
        min_csep = np.inf
        for i in range(len(centroids)):
            for j in range(i + 1, len(centroids)):
                d = np.sum((centroids[i] - centroids[j]) ** 2)
                if d < min_csep:
                    min_csep = d
        if not np.isfinite(min_csep) or min_csep <= eps:
```

```
            return np.nan
        return compact / (X.shape[0] * min_csep)

    try:
        results["Dunn Index"] = {
            "score": dunn_index(X, labels),
            "range": "[0, ∞)",
            "better": "higher"
        }
    except Exception:
        results["Dunn Index"] = {"score": np.nan, "range": "[0, ∞)", "better": "higher"}

    try:
        results["Xie–Beni Index"] = {
            "score": xie_beni_index(X, labels),
            "range": "[0, ∞)",
            "better": "lower"
        }
    except Exception:
        results["Xie–Beni Index"] = {"score": np.nan, "range": "[0, ∞)", "better": "lower"}

    # Pretty print
    print("\n📊 Clustering Evaluation Results:")
    for metric, info in results.items():
        score = info['score']
        score_str = f"{score:.4f}" if isinstance(score, (int, float)) and np.isfinite(score) else "NaN"
        print(f"{metric}: {score_str} --------- Range: {info['range']} ---------- {info['better'].capitalize()} is better")

    # Final cluster summary in one line
    cluster_summary = ", ".join([f"{c}:{cnt}" for c, cnt in zip(unique_clusters, counts)])
    print(f"📦 Total Clusters: {n_clusters} | Cluster Sizes: {cluster_summary}")

    return results if return_dict else None


# # --- Evaluate KMeans ---
# for df, name in all_dfs:
#     print(f"\nEvaluating {name} (KMeans)")
#     evaluate_clusters(
#         df,
#         df_name=name,
#         sil_metric="euclidean",
#         sample_size=None,
#         return_dict=False
#     )
```

combined evaluation function

```
import numpy as np
from collections import Counter
from scipy.spatial.distance import cdist
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score

# ==============================
```

```
# Config: metric order & weights
# ==============================
METRIC_ORDER = ["balance", "silhouette", "calinski_harabasz", "dunn", "davies_bouldin", "xie_beni"]
METRIC_WEIGHTS = {
    "balance": 0.35,
    "silhouette": 0.25,
    "calinski_harabasz": 0.20,
    "dunn": 0.10,
    "davies_bouldin": 0.05,
    "xie_beni": 0.05,
}

# ==================================
# Utilities: detect label column/name
# ==================================
def get_algo_and_label(df_name: str):
    if df_name.startswith("dbscan"):
        return "dbscan", "dbscan_cluster"
    if df_name.startswith("kmeans"):
        return "kmeans", "kmeans_cluster"
    if df_name.startswith("agg"):
        return "agg", "agg_cluster"
    if df_name.startswith("spc"):
        return "spc", "spc_cluster"
    raise ValueError(f"Unknown algorithm for {df_name}")

# ========================
# Metric helper calculations
# ========================
def _safe_min_pairwise_dist(M):
    if M.size == 0:
        return np.nan
    mask = ~np.eye(M.shape[0], dtype=bool)
    vals = M[mask]
    vals = vals[np.isfinite(vals)]
    if vals.size == 0:
        return np.nan
    return np.min(vals)

def _dunn_index(X, labels):
    uniq = np.unique(labels)
    if uniq.size < 2:
        return np.nan
    intramax = 0.0
    for k in uniq:
        pts = X[labels == k]
        if pts.shape[0] < 2:
            continue
        D = cdist(pts, pts)
        intramax = max(intramax, np.max(D))
    if not np.isfinite(intramax) or intramax <= 0:
        return np.nan
    inter_min = np.inf
    for i in range(uniq.size):
        for j in range(i+1, uniq.size):
            Xi = X[labels == uniq[i]]
```

```
                Xj = X[labels == uniq[j]]
                if Xi.size == 0 or Xj.size == 0:
                    continue
                D = cdist(Xi, Xj)
                m = np.min(D) if D.size else np.inf
                inter_min = min(inter_min, m)
        if not np.isfinite(inter_min) or inter_min <= 0:
            return np.nan
        return inter_min / intramax

    def _xie_beni_inv(X, labels):
        uniq = np.unique(labels)
        if uniq.size < 2:
            return np.nan
        cents = np.array([X[labels == k].mean(axis=0) for k in uniq])
        compact = 0.0
        for i, k in enumerate(uniq):
            pts = X[labels == k]
            if pts.size == 0:
                continue
            diff = pts - cents[i]
            compact += np.sum(diff * diff)
        C = cdist(cents, cents)
        min_csep = _safe_min_pairwise_dist(C)
        if not np.isfinite(min_csep) or min_csep <= 0:
            return np.nan
        xb = compact / (X.shape[0] * (min_csep ** 2))
        return 1.0 / (1.0 + xb)

    def _cluster_balance(labels, k_penalty=True):
        counts = np.array(list(Counter(labels).values()), dtype=float)
        K = counts.size
        if K < 2:
            return 0.0
        props = counts / counts.sum()
        base = 1.0 - np.std(props)
        base = max(0.0, float(base))
        if not k_penalty:
            return base
        penalty = 1.0 / np.sqrt(K)
        return base * penalty

    # ==========================================
    # Module 1: Compute metrics for a single frame
    # ==========================================
    def compute_all_metrics_for_df(df, df_name):
        algo, label_col = get_algo_and_label(df_name)
        labels_all = df[label_col].values
        X_all = df.drop(columns=[label_col]).values

        balance = _cluster_balance(labels_all, k_penalty=True)

        if algo == "dbscan":
            mask = labels_all != -1
            if mask.sum() < 2 or len(np.unique(labels_all[mask])) < 2:
                return {
```

```
                    "balance": balance,
                    "silhouette": np.nan,
                    "calinski_harabasz": np.nan,
                    "davies_bouldin": np.nan,
                    "dunn": np.nan,
                    "xie_beni": np.nan,
                }
            X = X_all[mask]
            labels = labels_all[mask]
        else:
            X = X_all
            labels = labels_all
            if len(np.unique(labels)) < 2:
                return {
                    "balance": balance,
                    "silhouette": np.nan,
                    "calinski_harabasz": np.nan,
                    "davies_bouldin": np.nan,
                    "dunn": np.nan,
                    "xie_beni": np.nan,
                }

        def _try(fn):
            try:
                v = fn()
                return float(v) if np.isfinite(v) else np.nan
            except Exception:
                return np.nan

        sil = _try(lambda: silhouette_score(X, labels))
        ch = _try(lambda: calinski_harabasz_score(X, labels))
        db_inv = _try(lambda: 1.0 / (1.0 + davies_bouldin_score(X, labels)))
        dunn = _try(lambda: _dunn_index(X, labels))
        xb_inv = _try(lambda: _xie_beni_inv(X, labels))

        return {
            "balance": balance,
            "silhouette": sil,
            "calinski_harabasz": ch,
            "davies_bouldin": db_inv,
            "dunn": dunn,
            "xie_beni": xb_inv,
        }

    # ========================================
    # Module 2: Evaluate all and store results
    # ========================================
    all_evaluation_results = {}
    metric_lists = {m: [] for m in METRIC_ORDER}
    composite_list = []

    def evaluate_all_datasets(all_dfs):
        for df, df_name in all_dfs:
            metrics = compute_all_metrics_for_df(df, df_name)
            all_evaluation_results[df_name] = metrics
            for m in METRIC_ORDER:
```

```
            metric_lists[m].append(metrics[m])

# ==================================
# Module 3: Ranking & final composite
# ==================================
def _sort_key_nan_last(val):
    return (-np.inf if np.isnan(val) else val)

def rank_per_metric(results_dict, metric):
    items = []
    for name, mdict in results_dict.items():
        v = mdict.get(metric, np.nan)
        items.append((name, v))
    items.sort(key=lambda x: _sort_key_nan_last(x[1]), reverse=True)
    return items

def composite_ranking(results_dict):
    per_metric_sorted = {m: rank_per_metric(results_dict, m) for m in METRIC_ORDER}
    n = len(results_dict)
    scores = {name: 0.0 for name in results_dict.keys()}
    for metric in METRIC_ORDER:
        weight = METRIC_WEIGHTS[metric]
        ranked_names = [name for name, _ in per_metric_sorted[metric]]
        for pos, name in enumerate(ranked_names):
            pts = (n - pos)
            val = results_dict[name].get(metric, np.nan)
            if np.isnan(val):
                pts = 0
            scores[name] += weight * pts
    final = sorted(scores.items(), key=lambda x: x[1], reverse=True)
    for name, score in final:
        composite_list.append(score)
    return per_metric_sorted, final

# ============================
# Module 4: Pretty-printing
# ============================
def print_rankings(per_metric_sorted, final_ranking, results_dict):
    print("\n📊 Ranked list for each metric:\n")
    for metric in METRIC_ORDER:
        print(f"--- {metric.upper()} ---")
        for name, _ in per_metric_sorted[metric]:
            val = results_dict[name][metric]
            if np.isnan(val):
                print(f"{name}: NaN")
            else:
                print(f"{name}: {val:.4f}")
        print()

    print("✅ Final Composite Ranking:\n")
    for i, (name, score) in enumerate(final_ranking, start=1):
        print(f"{i:>2}. {name}: Composite Score = {score:.4f}")

# =================
# Module 5: Run all
# =================
```

```
# Expect: all_dfs = [(df, "df_name"), ...] to be defined in your session.
evaluate_all_datasets(all_dfs)
per_metric_sorted, final_ranking = composite_ranking(all_evaluation_results)

# print_rankings(per_metric_sorted, final_ranking, all_evaluation_results)

# ✅ metric_lists contains per-metric values for all datasets
# ✅ composite_list contains the final composite scores



# silhouette_ranking = per_metric_sorted["silhouette"]
# final_ranking
```

sorting dfs based on combined values

```
# Create a dictionary mapping dataframe names to their composite scores
# final_ranking is a list of (name, score) tuples, sorted descending
score_map = dict(final_ranking)

# Sort all_dfs based on the composite scores
all_dfs_sorted = sorted(
    all_dfs,
    key=lambda x: score_map.get(x[1], -np.inf),  # x[1] is df_name, use -inf for missing names
    reverse=True  # highest combined score first
)
```

bar plot to show combined value

```
import matplotlib.pyplot as plt

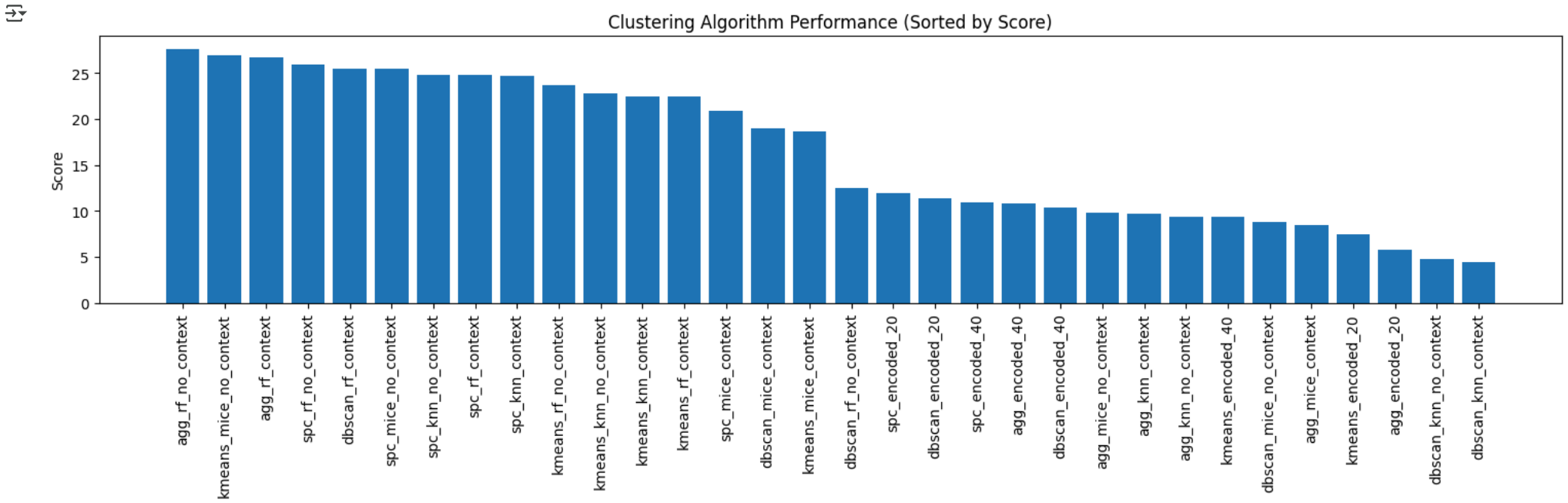
def plot_clustering_results(results_dict):
    """
    Generates a bar chart from clustering evaluation results provided as a dictionary.

    Args:
        results_dict (dict): A dictionary where keys are dataframe names and values are the scores.
    """
    # Extract names and scores from the dictionary
    names = list(results_dict.keys())
    scores = list(results_dict.values())

    # Create the bar chart
    plt.figure(figsize=(15, 5))
    plt.bar(names, scores)
    plt.xticks(rotation=90)
    plt.ylabel("Score")
    plt.title("Clustering Algorithm Performance (Sorted by Score)")
    plt.tight_layout()
    plt.show()
```

```
# Example usage (you can call this function with your results_dict)
# plot_clustering_results(my_results_dict)


# Example usage (you can call this function with your sorted_results)
plot_clustering_results(score_map)
```



cluster visualizing function

```
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
import seaborn as sns
import numpy as np

def visualize_clusters(dfs_list, cols=4, random_state=42):
    """
    Visualize multiple clustered DataFrames in a grid of PCA-reduced 2D scatter plots.

    dfs_list: list of tuples like (df, df_name)
    """
    rows = (len(dfs_list) + cols - 1) // cols
    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 4*rows))
    axes = axes.flatten()

    for ax, (df, df_name) in zip(axes, dfs_list):
        # Infer cluster column: first part of df_name + "_cluster"
        cluster_col = df_name.split("_")[0] + "_cluster"
```

```
        if cluster_col not in df.columns:
            print(f"⚠ Cluster column '{cluster_col}' not found in {df_name}, skipping...")
            continue

        # Reduce features to 2D for scatter plot
        X = df.drop(columns=[cluster_col]).values
        pca = PCA(n_components=2, random_state=random_state)
        X_2d = pca.fit_transform(X)

        # Compute cluster counts
        cluster_counts = df[cluster_col].value_counts()

        # Scatter plot with cluster colors
        sns.scatterplot(
            x=X_2d[:,0], y=X_2d[:,1],
            hue=df[cluster_col].astype(str),
            palette="tab10",
            ax=ax,
            legend=False,
            s=50
        )

        # Annotate cluster counts at centroids
        for cluster_id, count in cluster_counts.items():
            mask = df[cluster_col] == cluster_id
            centroid_x = X_2d[mask, 0].mean()
            centroid_y = X_2d[mask, 1].mean()
            ax.text(
                centroid_x, centroid_y,
                f"{cluster_id}: {count}",
                fontsize=9, fontweight='bold',
                ha='center', va='center',
                bbox=dict(facecolor='white', alpha=0.6, edgecolor='gray')
            )

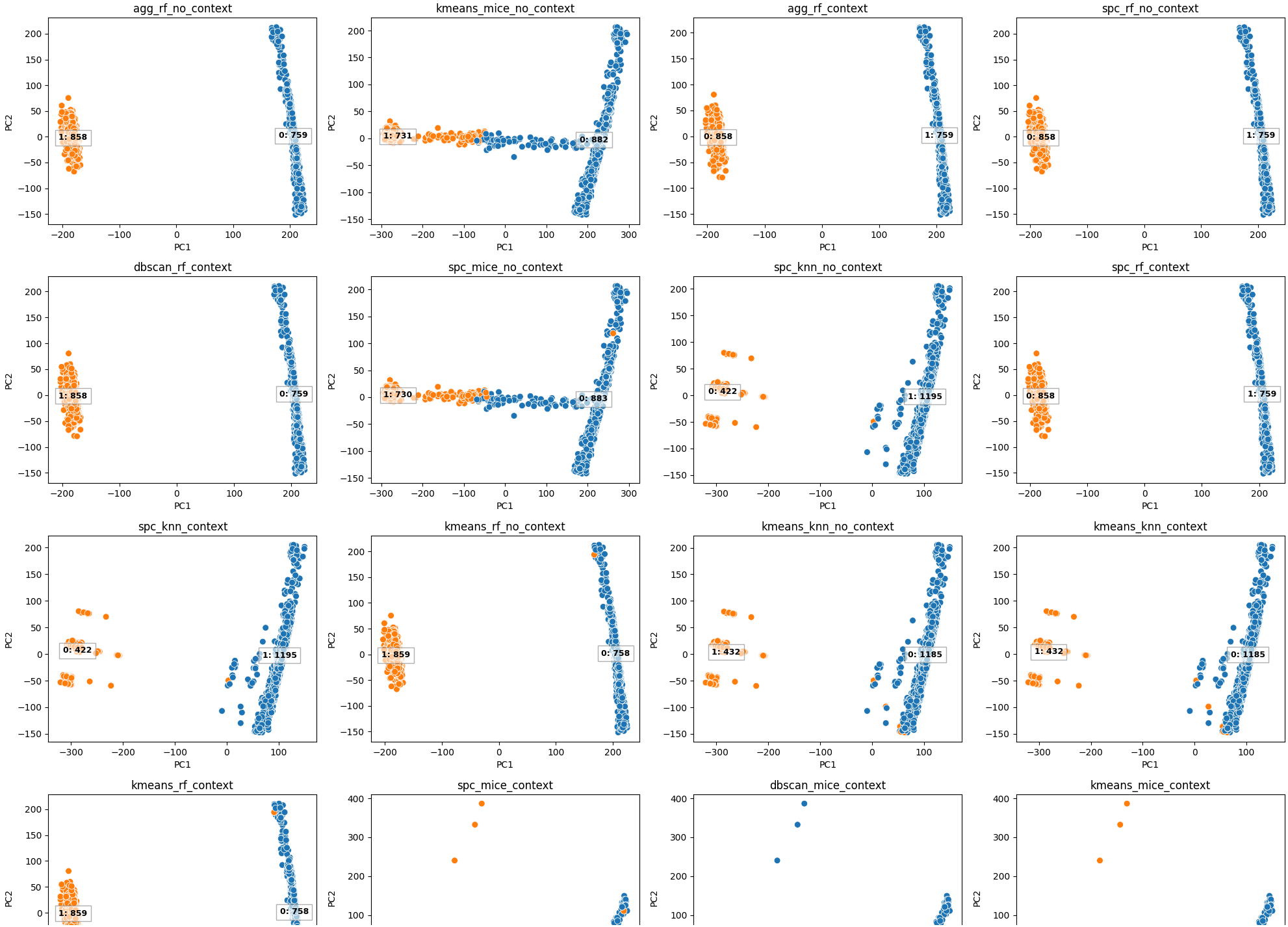
        ax.set_title(df_name)
        ax.set_xlabel("PC1")
        ax.set_ylabel("PC2")

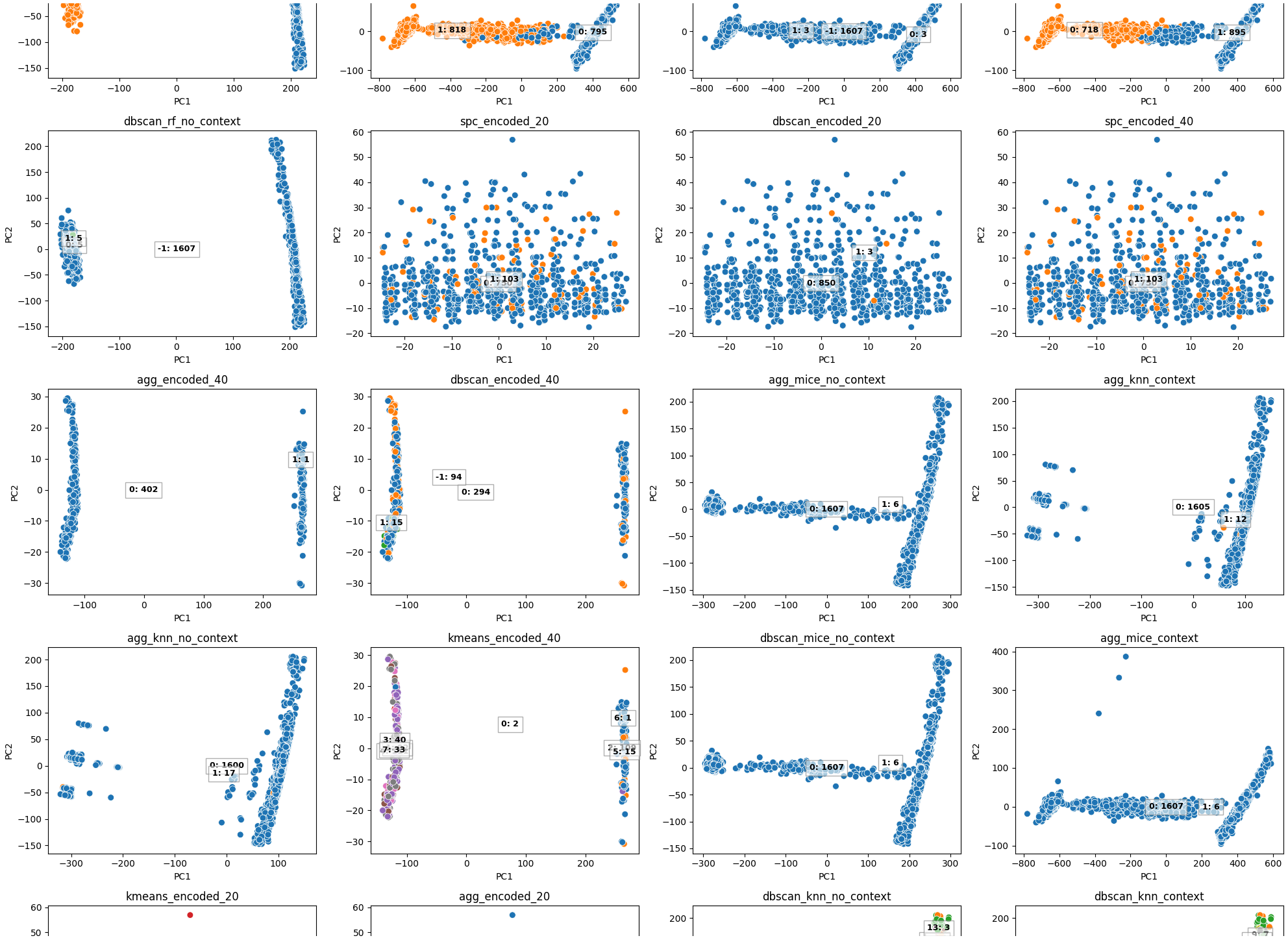
    # Remove unused axes
    for ax in axes[len(dfs_list):]:
        fig.delaxes(ax)

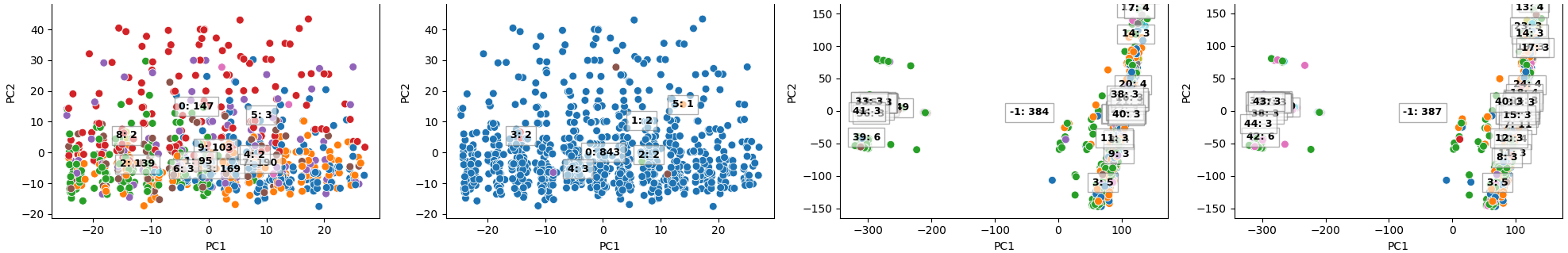
    plt.tight_layout()
    plt.show()
```

showing all 32 dataframe clusters

```
visualize_clusters(all_dfs_sorted, cols=4)
```


agg_rf_no_context
1: 858
0: 759
kmeans_mice_no_context
1: 731
0: 882
agg_rf_context
0: 858
1: 759
spc_rf_no_context
0: 858
1: 759
dbscan_rf_context
1: 858
0: 759
spc_mice_no_context
1: 730
0: 883
spc_knn_no_context
0: 422
1: 1195
spc_rf_context
0: 858
1: 759
spc_knn_context
0: 422
1: 1195
kmeans_rf_no_context
1: 859
0: 758
kmeans_knn_no_context
1: 432
0: 1185
kmeans_knn_context
1: 432
0: 1185
kmeans_rf_context
1: 859
0: 758
spc_mice_context
dbscan_mice_context
kmeans_mice_context
PC1
PC2


dbscan_rf_no_context
spc_encoded_20
dbscan_encoded_20
spc_encoded_40
agg_encoded_40
dbscan_encoded_40
agg_mice_no_context
agg_knn_context
agg_knn_no_context
kmeans_encoded_40
dbscan_mice_no_context
agg_mice_context
kmeans_encoded_20
agg_encoded_20
dbscan_knn_no_context
dbscan_knn_context
PC1
PC2


PC1
PC2
0: 147
5: 3
8: 2
9: 103
2: 139
4: 2
6: 3
169
PC1
PC2
5: 1
1: 2
3: 2
0: 843
2: 2
4: 3
PC1
PC2
-1: 384
14: 3
20: 4
38: 3
40: 3
11: 3
9: 3
3: 5
39: 6
PC1
PC2
-1: 387
40: 3
15: 3
12: 3
8: 3
3: 5
44: 3
42: 6

cluster label count showing function

```
import matplotlib.pyplot as plt

def plot_cluster_counts(valid_dfs_list, cols=4):
    all_counts = []

    for df, df_name in valid_dfs_list:
        # Infer cluster column name
        cluster_col = df_name.split("_")[0] + "_cluster"

        if cluster_col in df.columns:
            counts = df[cluster_col].value_counts().sort_index()
            all_counts.append((df_name, counts))
        else:
            print(f"⚠ Warning: Cluster column '{cluster_col}' not found in {df_name}")

    # Setup grid
    rows = (len(all_counts) + cols - 1) // cols
    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 4*rows))
    axes = axes.flatten()

    # Plot each cluster count
    for ax, (df_name, counts) in zip(axes, all_counts):
        counts.plot(kind="bar", ax=ax, color="skyblue", edgecolor="black")
        ax.set_title(df_name, fontsize=10)
        ax.set_xlabel("Cluster")
        ax.set_ylabel("Count")
        ax.tick_params(axis='x', rotation=0)

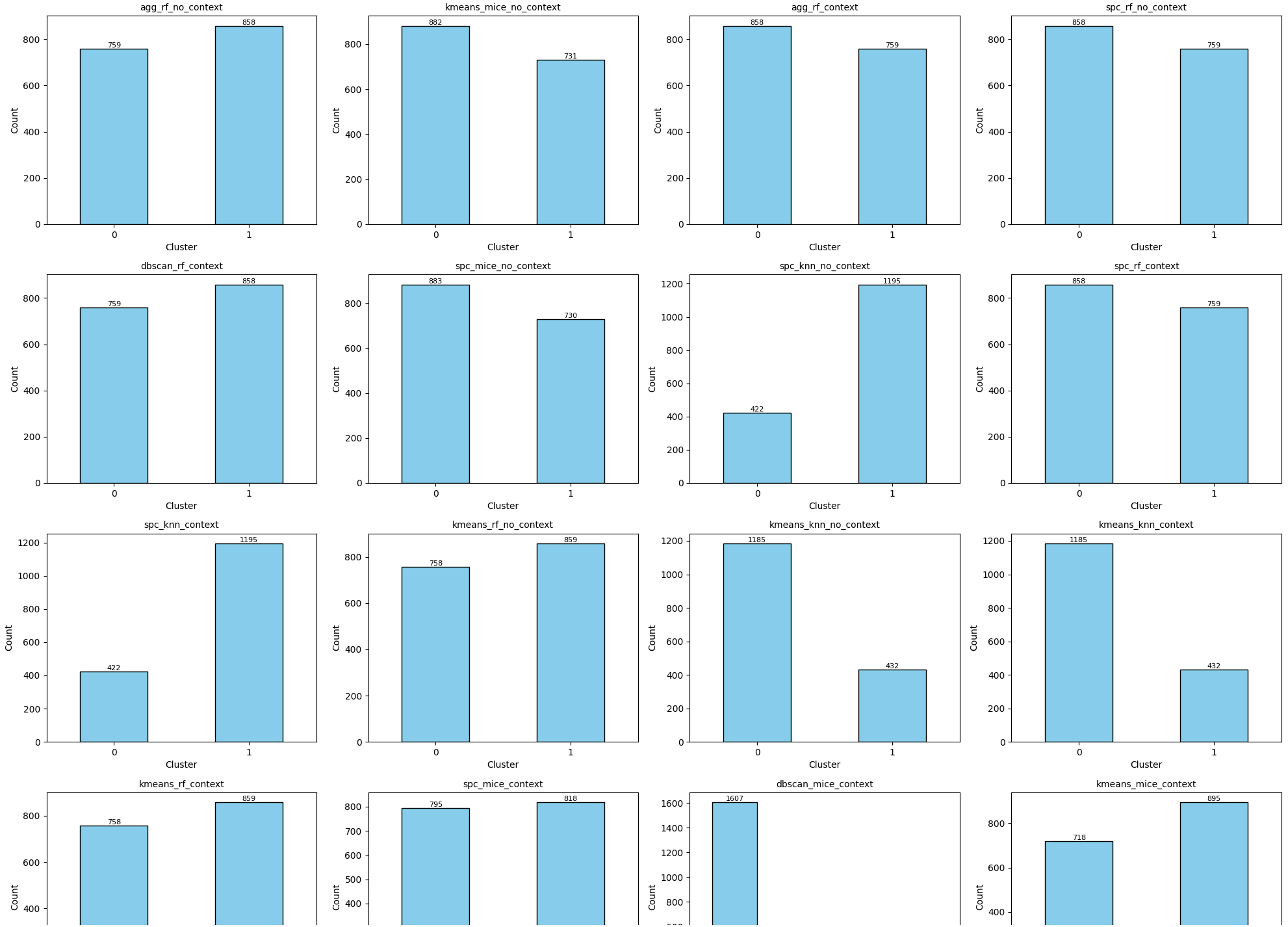
        # Annotate counts on top of bars
        for p in ax.patches:
            ax.annotate(
                str(int(p.get_height())),
                (p.get_x() + p.get_width() / 2, p.get_height()),
                ha="center", va="bottom", fontsize=8
            )

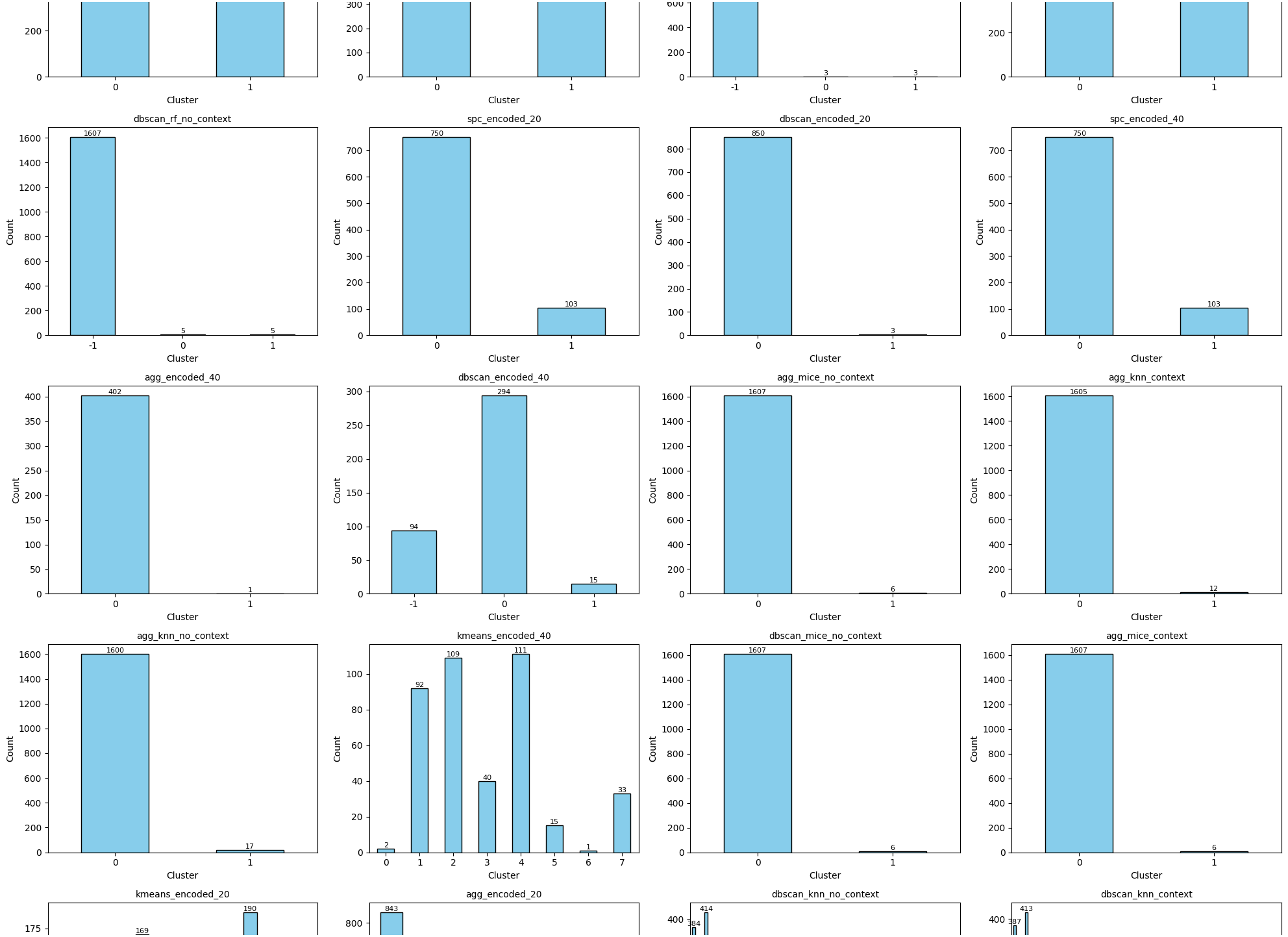
    # Remove unused axes
    for ax in axes[len(all_counts):]:
        fig.delaxes(ax)

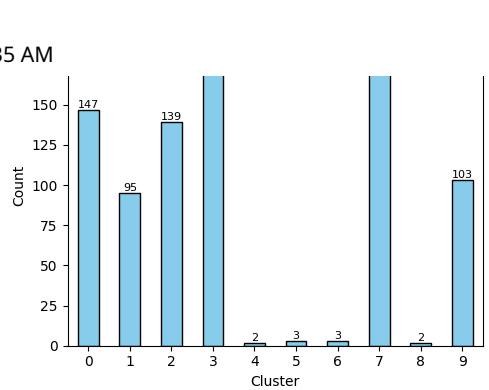
    plt.tight_layout()
    plt.show()

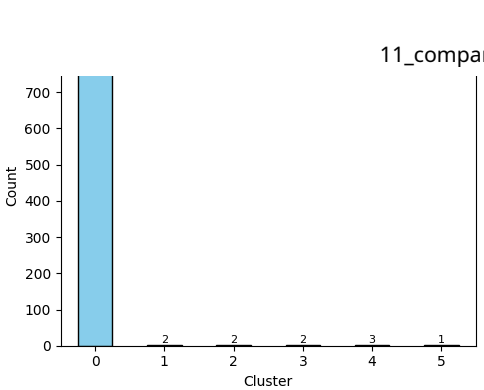


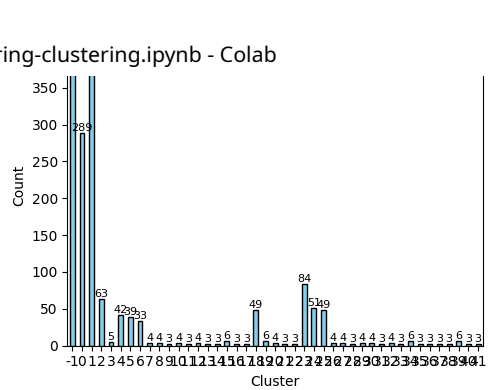
plot_cluster_counts(all_dfs_sorted, cols=4)
```

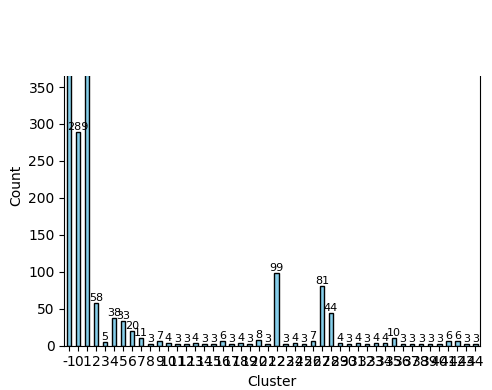

agg_rf_no_context
759
858
kmeans_mice_no_context
882
731
agg_rf_context
858
759
spc_rf_no_context
858
759
dbscan_rf_context
759
858
spc_mice_no_context
883
730
spc_knn_no_context
422
1195
spc_rf_context
858
759
spc_knn_context
422
1195
kmeans_rf_no_context
758
859
kmeans_knn_no_context
1185
432
kmeans_knn_context
1185
432
kmeans_rf_context
758
859
spc_mice_context
795
818
dbscan_mice_context
1607
kmeans_mice_context
718
895
Count
Cluster


dbscan_rf_no_context
Count
Cluster
1607
5
5
spc_encoded_20
750
103
dbscan_encoded_20
850
3
spc_encoded_40
750
103
agg_encoded_40
402
1
dbscan_encoded_40
94
294
15
agg_mice_no_context
1607
6
agg_knn_context
1605
12
agg_knn_no_context
1600
17
kmeans_encoded_40
2
92
109
40
111
15
1
33
dbscan_mice_no_context
1607
6
agg_mice_context
1607
6
kmeans_encoded_20
169
190
agg_encoded_20
843
dbscan_knn_no_context
384
414
dbscan_knn_context
387
413


Count
Cluster
147
95
139
2
3
3
2
103


Count
Cluster
2
2
2
3
1


Count
Cluster
289
63
42
39
33
49
84
54
49


Count
Cluster
289
58
38
33
20
99
81
44

metrics normalizer function

```
import numpy as np

def normalize_per_metric(per_metric_sorted):
    """
    Normalize all metric values in a per-metric dictionary using min-max scaling.

    Args:
        per_metric_sorted (dict): Dictionary like your example,
            {metric_name: [(dataset_name, value), ...]}

    Returns:
        dict: Normalized dictionary with same structure but values scaled [0, 1].
    """
    normalized_dict = {}

    for metric, values in per_metric_sorted.items():
        # Extract just the metric values
        vals = np.array([v for _, v in values], dtype=float)

        # Handle NaN values
        is_nan = np.isnan(vals)
        finite_vals = vals[~is_nan]

        if finite_vals.size == 0:
            # If all are NaN, keep as NaN
            normalized_vals = vals
        else:
            min_val = finite_vals.min()
            max_val = finite_vals.max()
            # Avoid division by zero if all values are equal
            if max_val == min_val:
                normalized_vals = np.ones_like(vals)
            else:
                normalized_vals = (vals - min_val) / (max_val - min_val)

        # Recreate the list of tuples
        normalized_dict[metric] = [(name, float(val)) for (name, _), val in zip(values, normalized_vals)]

    return normalized_dict


normalized_metrics = normalize_per_metric(per_metric_sorted)
```

metric comparsion between dfs

```
import matplotlib.pyplot as plt
import math

def plot_normalized_metrics(normalized_metrics):
```

```
        """
        Plot vertical bar charts for normalized metric values for all datasets,
        arranged in 2 columns.

        Args:
            normalized_metrics (dict): {metric_name: [(dataset_name, normalized_value), ...]}
        """
        metrics = list(normalized_metrics.keys())
        n_metrics = len(metrics)

        # Layout: 2 columns
        n_cols = 1
        n_rows = math.ceil(n_metrics / n_cols)

        plt.figure(figsize=(15*n_cols, 5*n_rows))

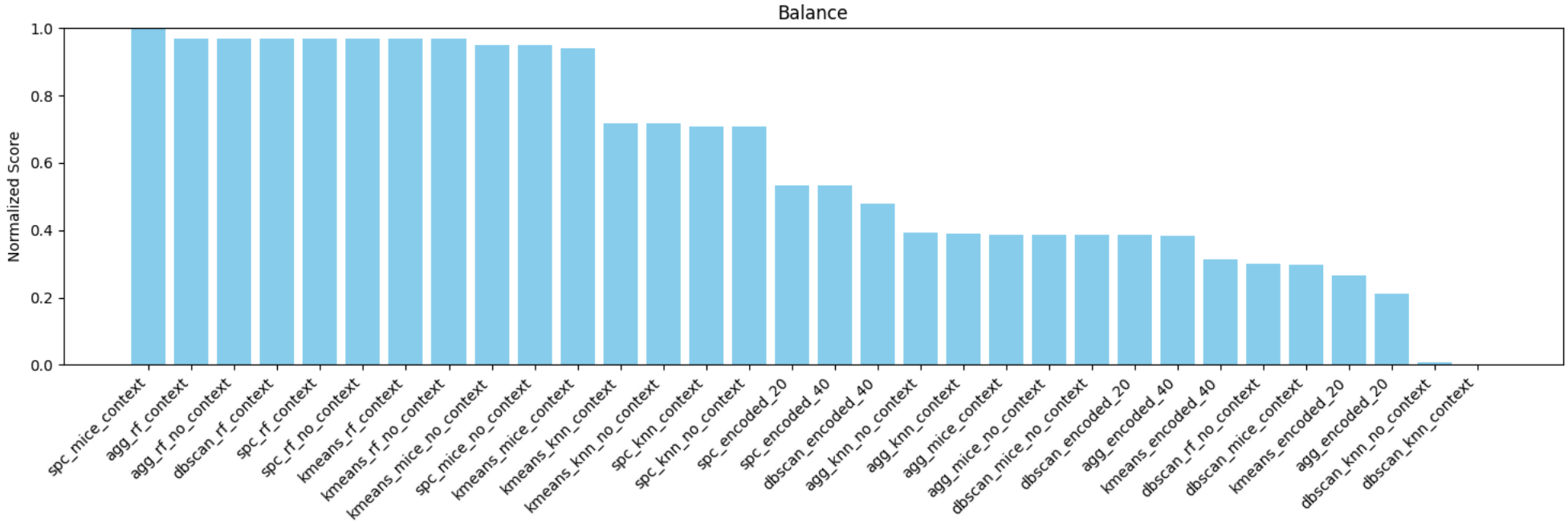
        for idx, metric in enumerate(metrics, start=1):
            plt.subplot(n_rows, n_cols, idx)

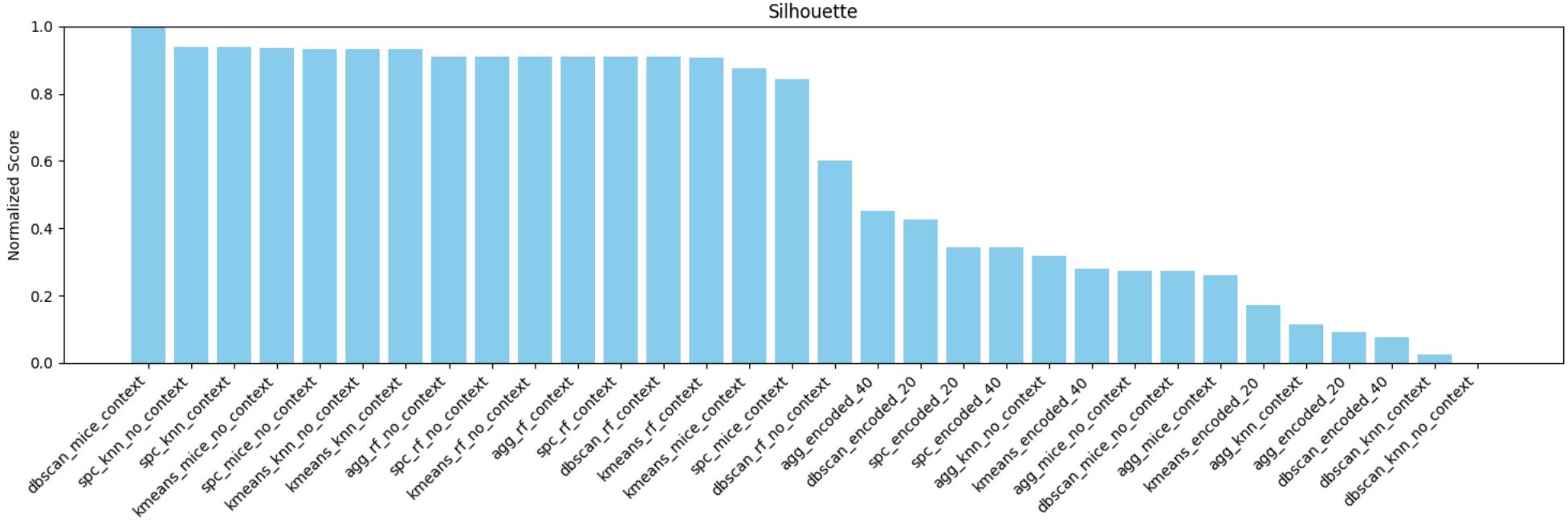
            data = normalized_metrics[metric]
            names = [name for name, _ in data]
            values = [val for _, val in data]

            # Vertical bar plot
            plt.bar(names, values, color='skyblue')
            plt.ylabel("Normalized Score")
            plt.title(metric.replace('_', ' ').title())
            plt.ylim(0, 1)  # normalized values between 0 and 1
            plt.xticks(rotation=45, ha='right')

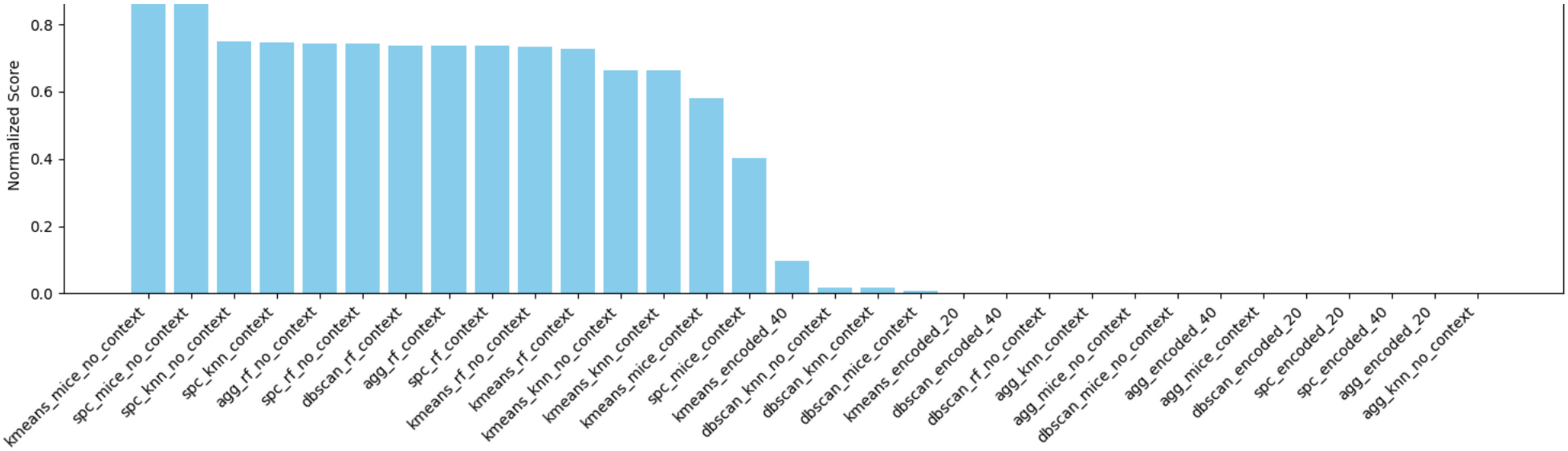
        plt.tight_layout()
        plt.show()

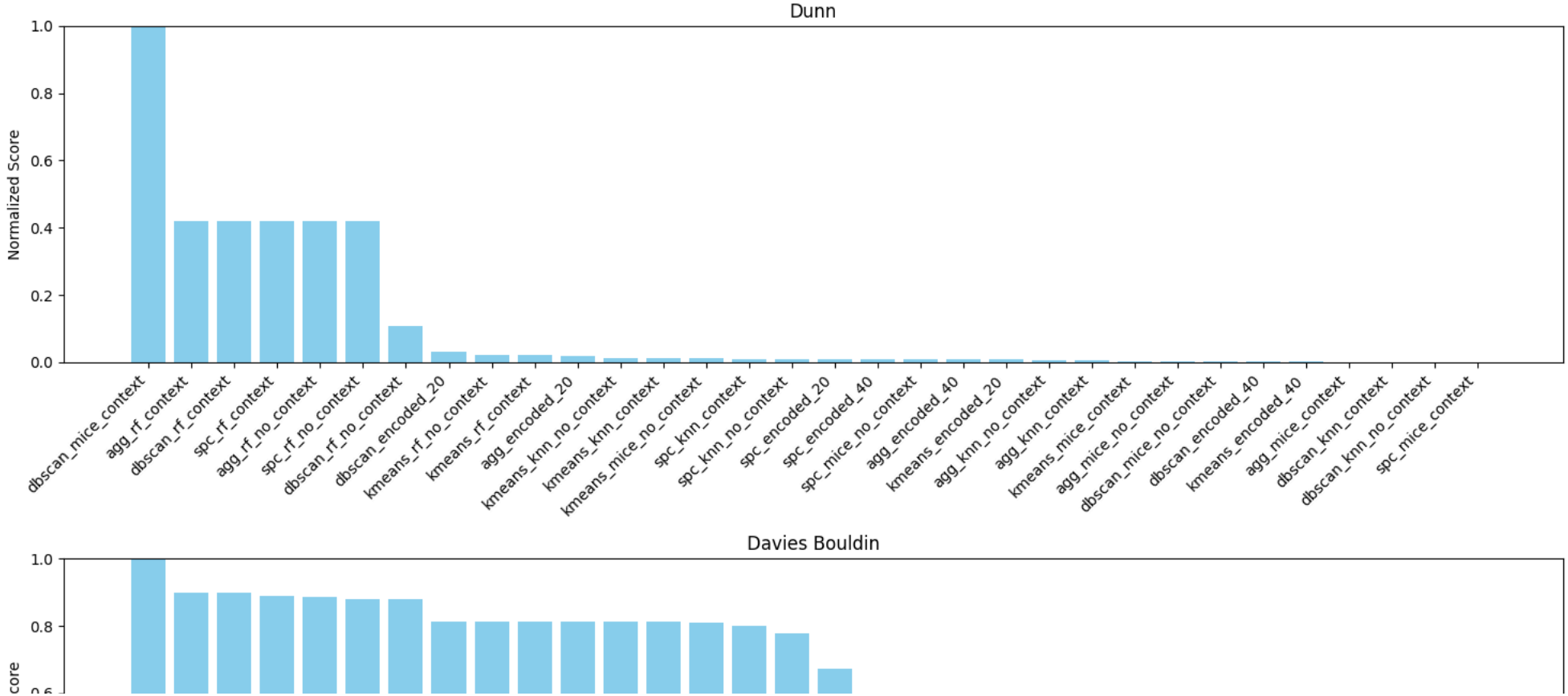

    plot_normalized_metrics(normalized_metrics)
```


Balance
Normalized Score
1.0
0.8
0.6
0.4
0.2
0.0
spc_mice_context
agg_rf_context
agg_rf_no_context
dbscan_rf_context
spc_rf_context
spc_rf_no_context
kmeans_rf_context
kmeans_rf_no_context
kmeans_mice_no_context
spc_mice_no_context
kmeans_mice_context
kmeans_knn_context
kmeans_knn_no_context
spc_knn_context
spc_knn_no_context
spc_encoded_20
spc_encoded_40
dbscan_encoded_40
agg_knn_no_context
agg_knn_context
agg_mice_context
agg_mice_no_context
dbscan_mice_no_context
dbscan_encoded_20
agg_encoded_40
kmeans_encoded_40
dbscan_rf_no_context
dbscan_mice_context
kmeans_encoded_20
agg_encoded_20
dbscan_knn_no_context
dbscan_knn_context

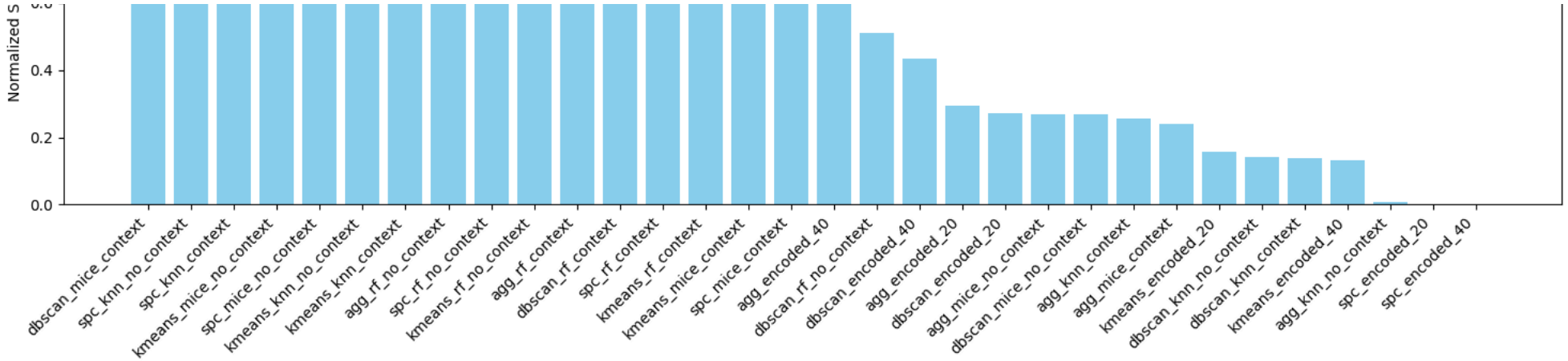

Silhouette
Normalized Score
1.0
0.8
0.6
0.4
0.2
0.0
dbscan_mice_context
spc_knn_no_context
spc_knn_context
kmeans_mice_no_context
spc_mice_no_context
kmeans_knn_no_context
kmeans_knn_context
agg_rf_no_context
spc_rf_no_context
kmeans_rf_no_context
agg_rf_context
spc_rf_context
dbscan_rf_context
kmeans_rf_context
kmeans_mice_context
spc_mice_context
dbscan_rf_no_context
agg_encoded_40
dbscan_encoded_20
spc_encoded_20
spc_encoded_40
agg_knn_no_context
kmeans_encoded_40
agg_mice_no_context
dbscan_mice_no_context
agg_mice_context
kmeans_encoded_20
agg_knn_context
agg_encoded_20
dbscan_encoded_40
dbscan_knn_context
dbscan_knn_no_context

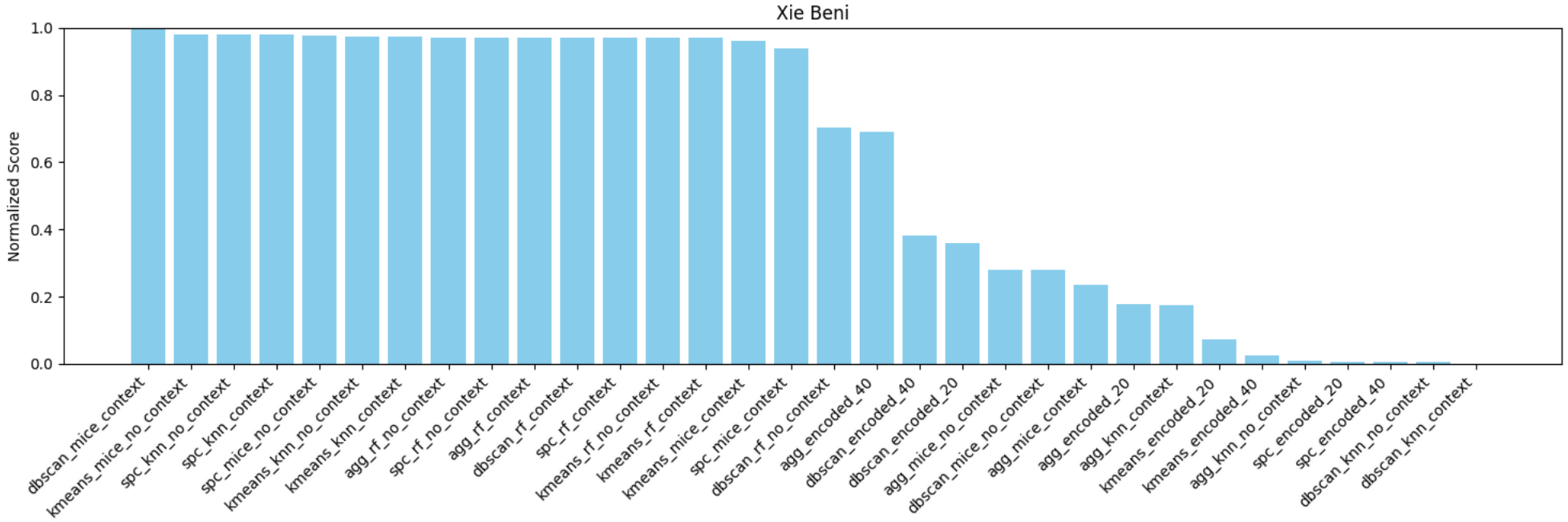

Calinski Harabasz
1.0


Normalized Score
0.0
0.2
0.4
0.6
0.8
kmeans_mice_no_context
spc_mice_no_context
spc_knn_no_context
spc_knn_context
agg_rf_no_context
spc_rf_no_context
dbscan_rf_context
agg_rf_context
spc_rf_context
kmeans_rf_no_context
kmeans_rf_context
kmeans_knn_no_context
kmeans_knn_context
kmeans_mice_context
spc_mice_context
kmeans_encoded_40
dbscan_knn_no_context
dbscan_knn_context
dbscan_mice_context
kmeans_encoded_20
dbscan_encoded_40
dbscan_rf_no_context
agg_knn_context
agg_mice_no_context
dbscan_mice_no_context
agg_encoded_40
agg_mice_context
dbscan_encoded_20
spc_encoded_20
spc_encoded_40
agg_encoded_20
agg_knn_no_context


Dunn
Normalized Score
0.0
0.2
0.4
0.6
0.8
1.0
dbscan_mice_context
agg_rf_context
dbscan_rf_context
spc_rf_context
agg_rf_no_context
spc_rf_no_context
dbscan_rf_no_context
dbscan_encoded_20
kmeans_rf_no_context
kmeans_rf_context
agg_encoded_20
kmeans_knn_no_context
kmeans_knn_context
kmeans_mice_no_context
spc_knn_context
spc_knn_no_context
spc_encoded_20
spc_encoded_40
spc_mice_no_context
agg_encoded_40
kmeans_encoded_20
agg_knn_no_context
agg_knn_context
kmeans_mice_context
agg_mice_no_context
dbscan_mice_no_context
dbscan_encoded_40
kmeans_encoded_40
agg_mice_context
dbscan_knn_context
dbscan_knn_no_context
spc_mice_context

Davies Bouldin
1.0
0.8


Normalized S
0.0
0.2
0.4
dbscan_mice_context
spc_knn_no_context
spc_knn_context
kmeans_mice_no_context
spc_mice_no_context
kmeans_knn_no_context
kmeans_knn_context
agg_rf_no_context
spc_rf_no_context
kmeans_rf_no_context
agg_rf_context
dbscan_rf_context
spc_rf_context
kmeans_rf_context
kmeans_mice_context
spc_mice_context
agg_encoded_40
dbscan_rf_no_context
dbscan_encoded_40
agg_encoded_20
dbscan_encoded_20
agg_mice_no_context
dbscan_mice_no_context
agg_knn_context
agg_mice_context
kmeans_encoded_20
dbscan_knn_no_context
dbscan_knn_context
kmeans_encoded_40
agg_knn_no_context
spc_encoded_20
spc_encoded_40


Xie Beni
Normalized Score
0.0
0.2
0.4
0.6
0.8
1.0
dbscan_mice_context
kmeans_mice_no_context
spc_knn_no_context
spc_knn_context
spc_mice_no_context
kmeans_knn_no_context
kmeans_knn_context
agg_rf_no_context
spc_rf_no_context
agg_rf_context
dbscan_rf_context
spc_rf_context
kmeans_rf_no_context
kmeans_rf_context
kmeans_mice_context
spc_mice_context
dbscan_rf_no_context
agg_encoded_40
dbscan_encoded_40
dbscan_encoded_20
agg_mice_no_context
dbscan_mice_no_context
agg_mice_context
agg_encoded_20
agg_knn_context
kmeans_encoded_20
kmeans_encoded_40
agg_knn_no_context
spc_encoded_20
spc_encoded_40
dbscan_knn_no_context
dbscan_knn_context

```
def remove_dataset_from_per_metric(per_metric_sorted, dataset_name="dbscan_rf_no_context"):
    """
    Remove a specific dataset from per_metric_sorted dictionary.

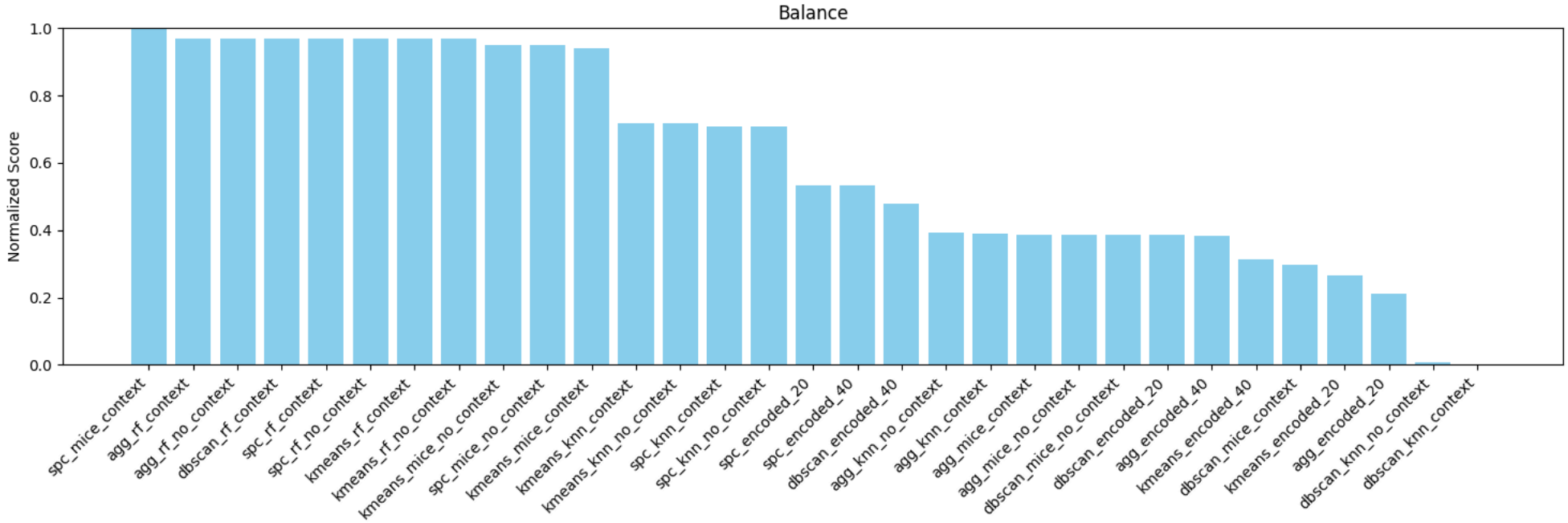
    Args:
        per_metric_sorted (dict): {metric_name: [(dataset_name, value), ...]}
        dataset_name (str): Name of the dataset to remove

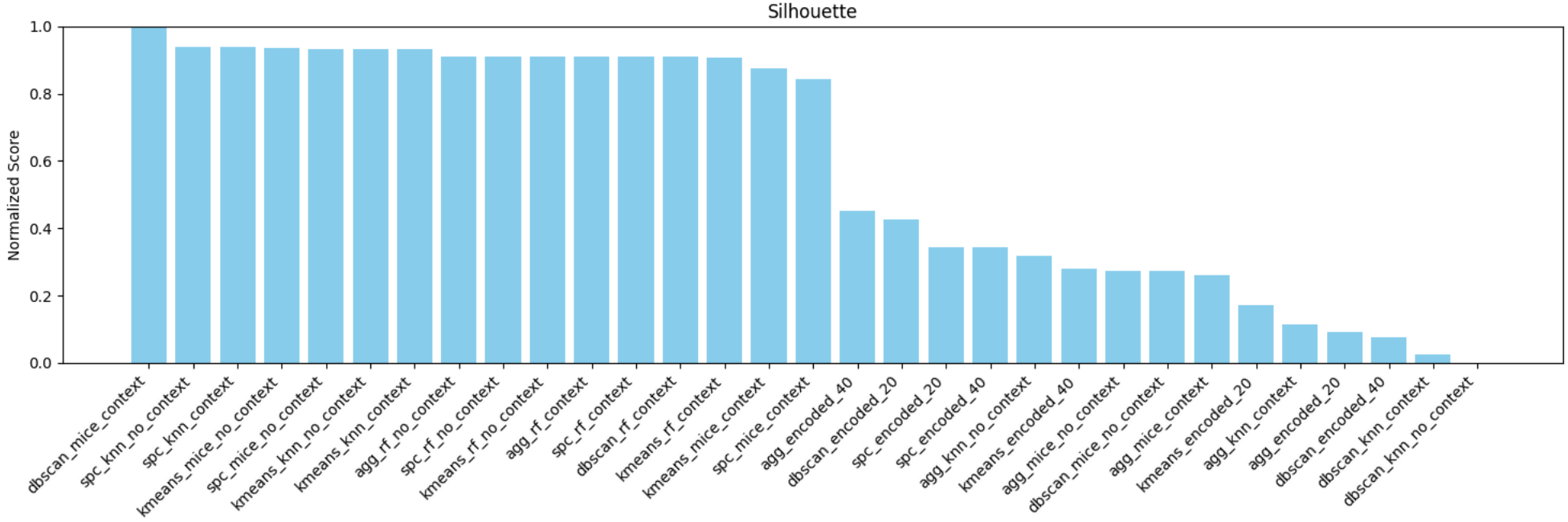
    Returns:
        dict: New dictionary with the dataset removed
    """
    new_dict = {}
    for metric, data in per_metric_sorted.items():
        new_data = [(name, val) for name, val in data if name != dataset_name]
        new_dict[metric] = new_data
    return new_dict


per_metric_sorted_clean = remove_dataset_from_per_metric(per_metric_sorted)

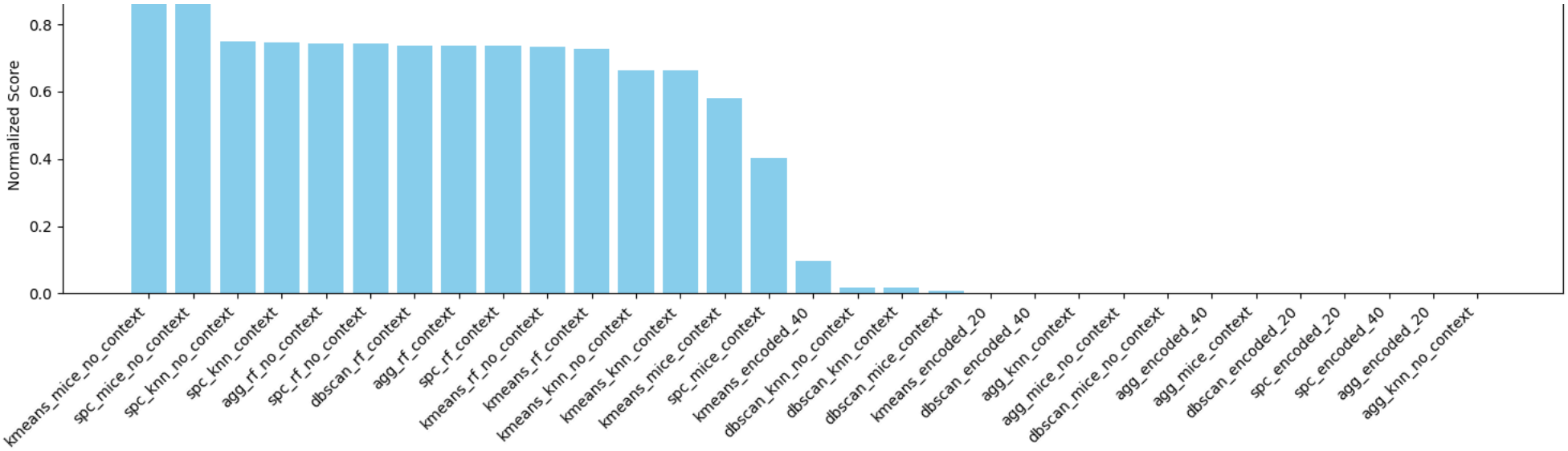

clean_normalized_metrics = normalize_per_metric(per_metric_sorted_clean)

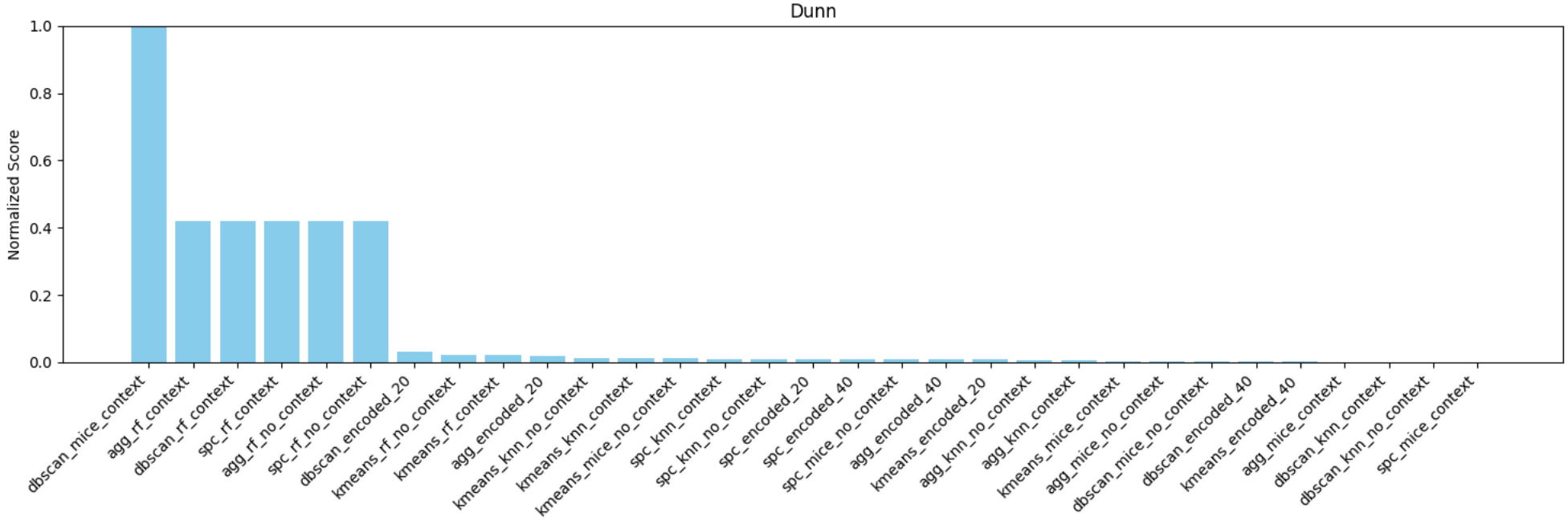

plot_normalized_metrics(clean_normalized_metrics)
```

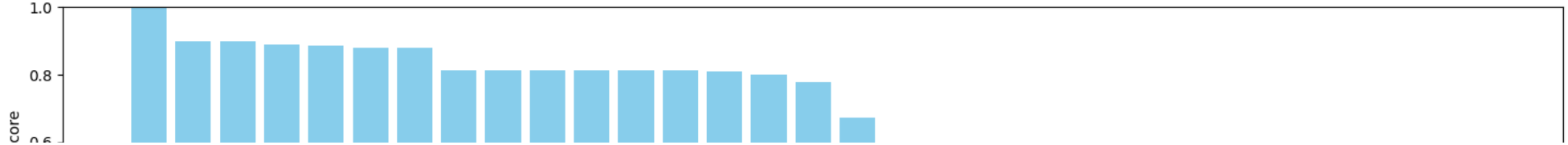

Balance
Normalized Score
1.0
0.8
0.6
0.4
0.2
0.0
spc_mice_context
agg_rf_context
agg_rf_no_context
dbscan_rf_context
spc_rf_context
spc_rf_no_context
kmeans_rf_context
kmeans_rf_no_context
kmeans_mice_no_context
spc_mice_no_context
kmeans_mice_context
kmeans_knn_context
kmeans_knn_no_context
spc_knn_context
spc_knn_no_context
spc_encoded_20
spc_encoded_40
dbscan_encoded_40
agg_knn_no_context
agg_knn_context
agg_mice_context
agg_mice_no_context
dbscan_mice_no_context
dbscan_encoded_20
agg_encoded_40
kmeans_encoded_40
dbscan_mice_context
kmeans_encoded_20
agg_encoded_20
dbscan_knn_no_context
dbscan_knn_context

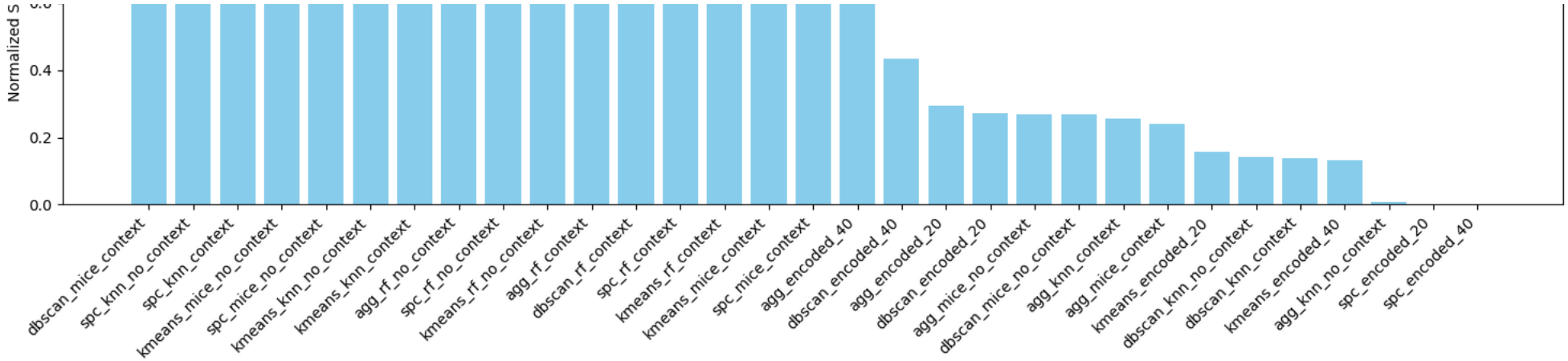

Silhouette
Normalized Score
1.0
0.8
0.6
0.4
0.2
0.0
dbscan_mice_context
spc_knn_no_context
spc_knn_context
kmeans_mice_no_context
spc_mice_no_context
kmeans_knn_no_context
kmeans_knn_context
agg_rf_no_context
spc_rf_no_context
kmeans_rf_no_context
agg_rf_context
spc_rf_context
dbscan_rf_context
kmeans_rf_context
kmeans_mice_context
spc_mice_context
agg_encoded_40
dbscan_encoded_20
spc_encoded_20
spc_encoded_40
agg_knn_no_context
kmeans_encoded_40
agg_mice_no_context
dbscan_mice_no_context
agg_mice_context
kmeans_encoded_20
agg_knn_context
agg_encoded_20
dbscan_encoded_40
dbscan_knn_context
dbscan_knn_no_context

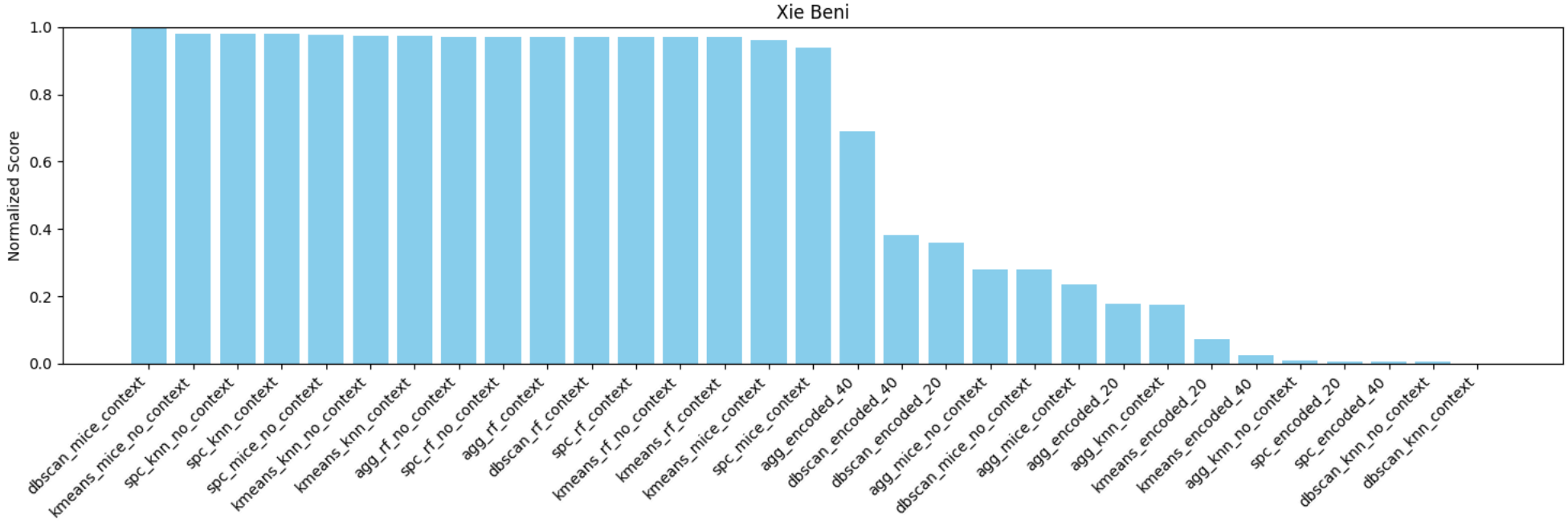

Calinski Harabasz
1.0


Normalized Score
0.0
0.2
0.4
0.6
0.8
kmeans_mice_no_context
spc_mice_no_context
spc_knn_no_context
spc_knn_context
agg_rf_no_context
spc_rf_no_context
dbscan_rf_context
agg_rf_context
spc_rf_context
kmeans_rf_no_context
kmeans_rf_context
kmeans_knn_no_context
kmeans_knn_context
kmeans_mice_context
spc_mice_context
kmeans_encoded_40
dbscan_knn_no_context
dbscan_knn_context
dbscan_mice_context
kmeans_encoded_20
dbscan_encoded_40
agg_knn_context
agg_mice_no_context
dbscan_mice_no_context
agg_encoded_40
agg_mice_context
dbscan_encoded_20
spc_encoded_20
spc_encoded_40
agg_encoded_20
agg_knn_no_context


Dunn
Normalized Score
0.0
0.2
0.4
0.6
0.8
1.0
dbscan_mice_context
agg_rf_context
dbscan_rf_context
spc_rf_context
agg_rf_no_context
spc_rf_no_context
dbscan_encoded_20
kmeans_rf_no_context
kmeans_rf_context
agg_encoded_20
kmeans_knn_no_context
kmeans_knn_context
kmeans_mice_no_context
spc_knn_context
spc_knn_no_context
spc_encoded_20
spc_encoded_40
spc_mice_no_context
agg_encoded_40
kmeans_encoded_20
agg_knn_no_context
agg_knn_context
kmeans_mice_context
agg_mice_no_context
dbscan_mice_no_context
dbscan_encoded_40
kmeans_encoded_40
agg_mice_context
dbscan_knn_context
dbscan_knn_no_context
spc_mice_context


Davies Bouldin
1.0
0.8


Normalized S
0.0
0.2
0.4
dbscan_mice_context
spc_knn_no_context
spc_knn_context
kmeans_mice_no_context
spc_mice_no_context
kmeans_knn_no_context
kmeans_knn_context
agg_rf_no_context
spc_rf_no_context
kmeans_rf_no_context
agg_rf_context
dbscan_rf_context
spc_rf_context
kmeans_rf_context
kmeans_mice_context
spc_mice_context
agg_encoded_40
dbscan_encoded_40
agg_encoded_20
dbscan_encoded_20
agg_mice_no_context
dbscan_mice_no_context
agg_knn_context
agg_mice_context
kmeans_encoded_20
dbscan_knn_no_context
dbscan_knn_context
kmeans_encoded_40
agg_knn_no_context
spc_encoded_20
spc_encoded_40


Xie Beni
Normalized Score
0.0
0.2
0.4
0.6
0.8
1.0
dbscan_mice_context
kmeans_mice_no_context
spc_knn_no_context
spc_knn_context
spc_mice_no_context
kmeans_knn_no_context
kmeans_knn_context
agg_rf_no_context
spc_rf_no_context
agg_rf_context
dbscan_rf_context
spc_rf_context
kmeans_rf_no_context
kmeans_rf_context
kmeans_mice_context
spc_mice_context
agg_encoded_40
dbscan_encoded_40
dbscan_encoded_20
agg_mice_no_context
dbscan_mice_no_context
agg_mice_context
agg_encoded_20
agg_knn_context
kmeans_encoded_20
kmeans_encoded_40
agg_knn_no_context
spc_encoded_20
spc_encoded_40
dbscan_knn_no_context
dbscan_knn_context

Extra comparison: computing average metrics for groups to compare

```
import numpy as np

def compute_group_average_metrics(all_evaluation_results, df_groups):
    """
    Compute average metrics for each group of datasets.

    Args:
        all_evaluation_results (dict): {dataset_name: {metric_name: value, ...}}
        df_groups (dict): {group_name: list of (df, dataset_name)}

    Returns:
        dict: {group_name: {metric_name: average_value}}
    """
    group_avg_metrics = {}

    for group_name, df_list in df_groups.items():
        metric_sums = {}
        metric_counts = {}

        # Loop through each dataset in the group
        for df, dataset_name in df_list:
            metrics = all_evaluation_results.get(dataset_name)
            if metrics is None:
                continue

            for metric, value in metrics.items():
                if np.isnan(value):
                    continue
                metric_sums[metric] = metric_sums.get(metric, 0) + value
                metric_counts[metric] = metric_counts.get(metric, 0) + 1

        # Compute average
        group_avg_metrics[group_name] = {}
        for metric in metric_sums:
            group_avg_metrics[group_name][metric] = metric_sums[metric] / metric_counts[metric]

    return group_avg_metrics



# Define your groups as a dictionary
df_groups = {
    "agg": agg_dfs,
    "dbscan": dbscan_dfs,
    "kmeans": kmeans_dfs,
    "spc": spc_dfs,
    "no_context": no_context_dfs,
    "context": context_dfs,
    "no_imputed": no_imputed_dfs,
    "mice": mice_dfs,
    "rf": rf_dfs,
    "knn": knn_dfs,
```

```
}

group_avg_metrics = compute_group_average_metrics(all_evaluation_results, df_groups)
```

showing groups comparison bar chart

```
import matplotlib.pyplot as plt
import numpy as np

def compare_group_metrics(group_avg_metrics, selected_groups, width=14, height=5):
    """
    Compare metrics between multiple groups using horizontal grouped bar chart.

    Args:
        group_avg_metrics (dict): {group_name: {metric_name: average_value}}
        selected_groups (list): List of group names to compare (must be keys in group_avg_metrics)
    """
    metrics = list(next(iter(group_avg_metrics.values())).keys())  # list of metrics
    n_metrics = len(metrics)
    n_groups = len(selected_groups)

    # Bar positions
    y = np.arange(n_metrics)
    bar_height = 0.8 / n_groups  # split each metric bar

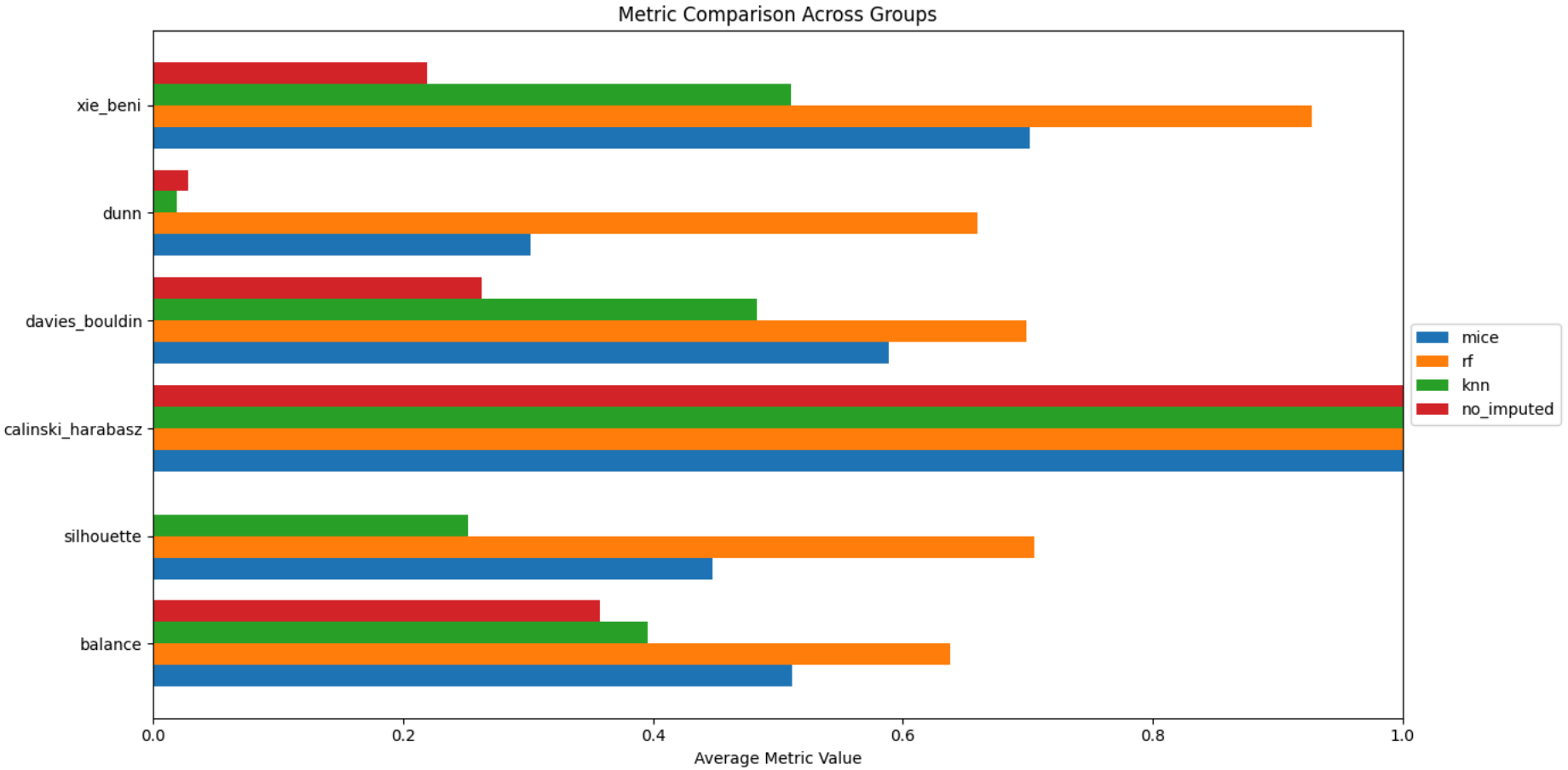
    plt.figure(figsize=(width, height))

    # Plot each group's bars
    for i, group in enumerate(selected_groups):
        values = [group_avg_metrics[group][metric] for metric in metrics]
        plt.barh(y + i*bar_height, values, height=bar_height, label=group)

    plt.yticks(y + bar_height*(n_groups-1)/2, metrics)  # center metric labels
    plt.xlabel("Average Metric Value")
    plt.title("Metric Comparison Across Groups")
    plt.legend(loc='center left', bbox_to_anchor=(1, 0.5))  # legend to the right
    plt.xlim(0, 1)  # assuming normalized metrics; adjust if not
    plt.tight_layout()
    plt.show()
```

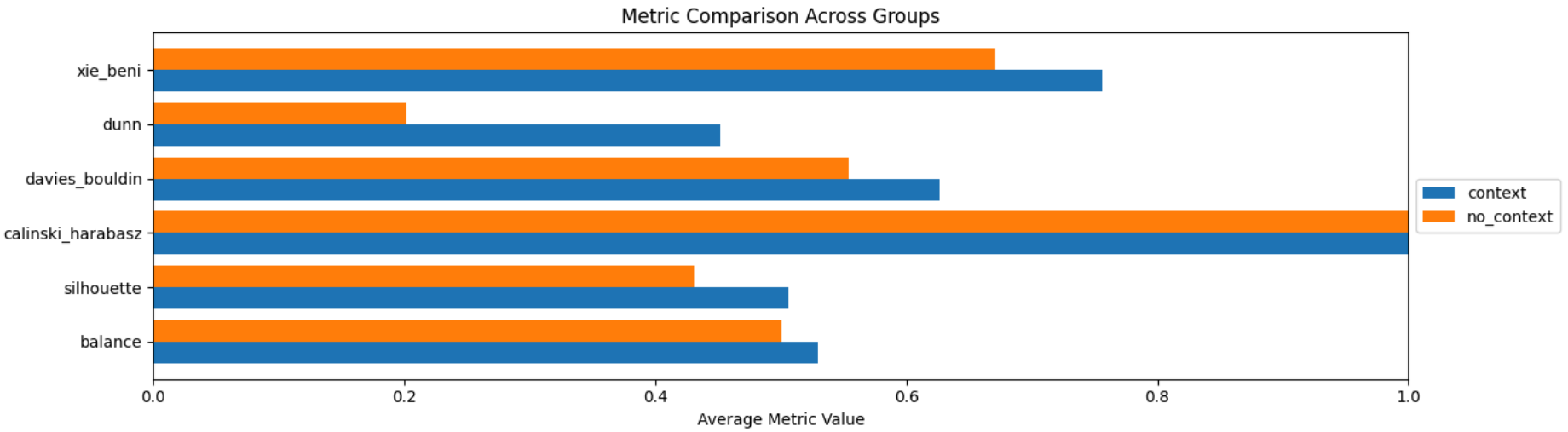
comparison between imputation

```
compare_group_metrics(group_avg_metrics, ["mice", "rf", "knn", "no_imputed"], height=7)
```



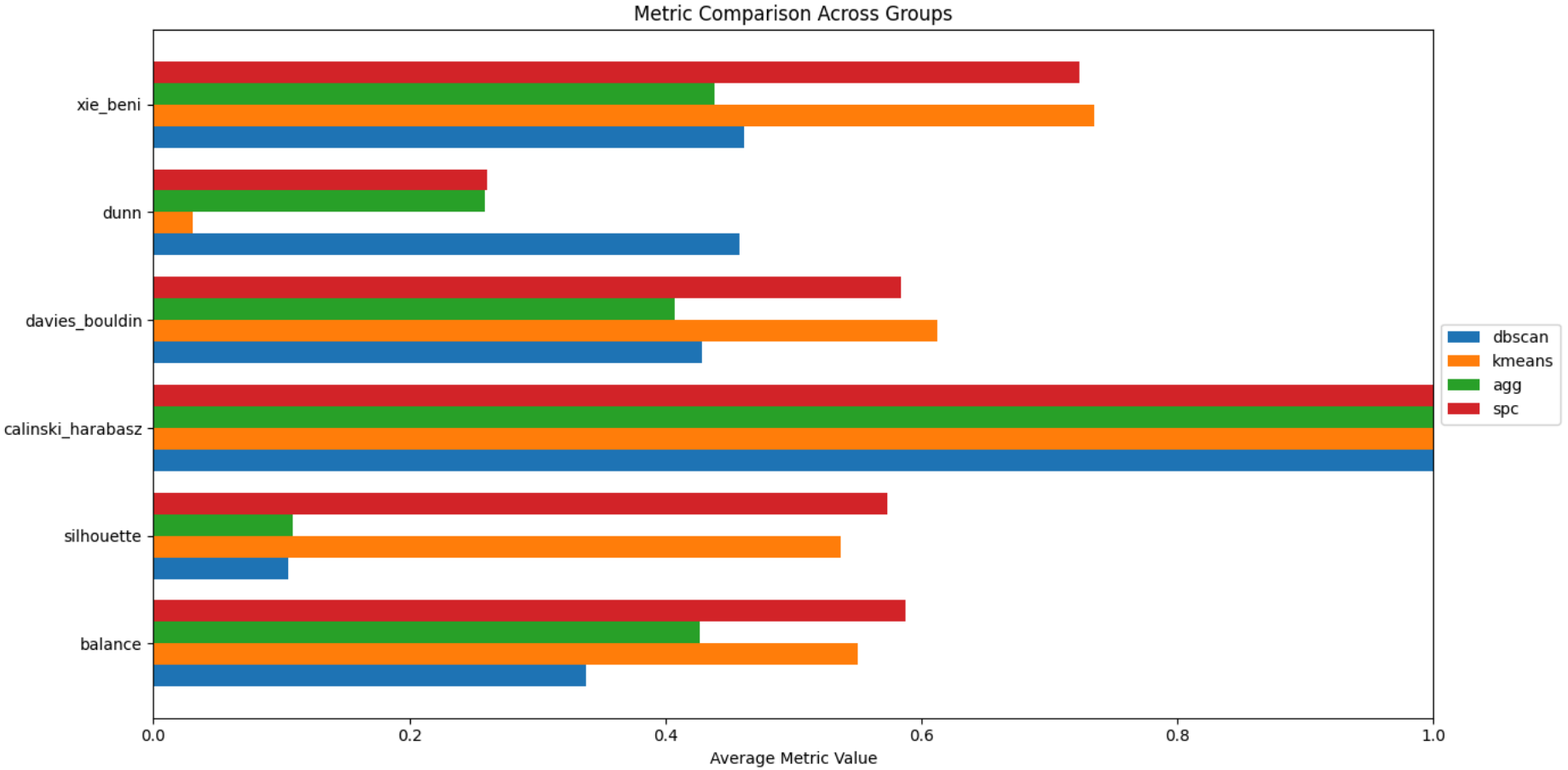
comparing contextual or non contextual imputaion

```
compare_group_metrics(group_avg_metrics, ["context", "no_context"], height=4)
```



comparing clustering alorthms

```
compare_group_metrics(group_avg_metrics, ["dbscan", "kmeans", "agg", "spc"], height=7)
```




```
import pandas as pd

def create_metrics_table(per_metric_sorted, score_map):
    """
    Creates a pandas DataFrame from the per_metric_sorted dictionary and sorts it by composite score.

    Args:
        per_metric_sorted (dict): {metric_name: [(dataset_name, value), ...]}
        score_map (dict): {dataset_name: composite_score}

    Returns:
        pd.DataFrame: A DataFrame with dataset names as index and metrics as columns, sorted by composite score.
    """
    # Get all unique dataset names
    dataset_names = sorted(list(set([name for metric_data in per_metric_sorted.values() for name, _ in metric_data])))

    # Create a dictionary to store metrics for each dataset
    data = {name: {} for name in dataset_names}
```

```
        # Populate the dictionary
        for metric, metric_data in per_metric_sorted.items():
            for name, value in metric_data:
                data[name][metric] = value

        # Create the DataFrame
        df_metrics = pd.DataFrame.from_dict(data, orient='index')

        # Sort columns based on METRIC_ORDER
        sorted_columns = [m for m in METRIC_ORDER if m in df_metrics.columns]
        df_metrics = df_metrics[sorted_columns]

        # Add composite score as a column for sorting and display
        df_metrics['composite_score'] = df_metrics.index.map(score_map)

        # Sort by composite score in descending order
        df_metrics = df_metrics.sort_values(by='composite_score', ascending=False)

        # The composite score column will now be kept in the output
        return df_metrics

    # Create and display the table, passing score_map
    df_metrics_table = create_metrics_table(per_metric_sorted, score_map)
    display(df_metrics_table)
```

	balance	silhouette	calinski_harabasz	dunn	davies_bouldin	xie_beni	composite_score
agg_rf_no_context	0.685461	0.756380	9792.497921	0.980954	0.730318	0.960545	27.65
kmeans_mice_no_context	0.674009	0.792085	13154.910479	0.030805	0.791936	0.970546	26.95
agg_rf_context	0.685461	0.754761	9686.611574	0.982102	0.729068	0.960131	26.70
spc_rf_no_context	0.685461	0.756380	9792.497921	0.980954	0.730318	0.960545	25.95
dbscan_rf_context	0.685461	0.754761	9686.611574	0.982102	0.729068	0.960131	25.50
spc_mice_no_context	0.673571	0.787801	12376.439745	0.021246	0.789193	0.968759	25.45
spc_knn_no_context	0.538092	0.795369	9875.866564	0.025660	0.800913	0.969425	24.80
spc_rf_context	0.685461	0.754761	9686.611574	0.982102	0.729068	0.960131	24.80
spc_knn_context	0.538092	0.794904	9833.815232	0.025660	0.800527	0.969298	24.70
kmeans_rf_no_context	0.685023	0.755367	9654.365485	0.055459	0.729813	0.960009	23.65
kmeans_knn_no_context	0.542465	0.785370	8744.775850	0.032461	0.786055	0.965104	22.75
kmeans_knn_context	0.542465	0.784890	8708.895980	0.032461	0.785642	0.964965	22.45
kmeans_rf_context	0.685023	0.753743	9549.182978	0.054177	0.728559	0.959586	22.45
spc_mice_context	0.702065	0.670781	5302.263750	0.001820	0.701281	0.929418	20.85
dbscan_mice_context	0.306703	0.874364	96.547161	2.327724	0.883905	0.989749	18.95
kmeans_mice_context	0.668310	0.708758	7649.178141	0.011035	0.718630	0.950553	18.70
dbscan_rf_no_context	0.307709	0.351424	4.619301	0.256321	0.481121	0.697853	12.45
spc_encoded_20	0.438937	0.012104	1.372098	0.025583	0.061092	0.014959	11.90
dbscan_encoded_20	0.356040	0.121859	1.690365	0.071764	0.284405	0.361745	11.40
spc_encoded_40	0.438937	0.012104	1.372098	0.025583	0.061092	0.014959	10.90
agg_encoded_40	0.355308	0.157153	2.160973	0.020834	0.613707	0.685253	10.80
dbscan_encoded_40	0.409134	-0.339146	8.739920	0.008877	0.418675	0.381333	10.40
agg_mice_no_context	0.356184	-0.077905	2.344871	0.009983	0.281811	0.282000	9.85
agg_knn_context	0.358801	-0.285265	2.609730	0.013441	0.272135	0.179908	9.65
kmeans_encoded_40	0.315207	-0.071859	1281.887066	0.008478	0.168569	0.032546	9.35
agg_knn_no_context	0.360987	-0.019878	0.278632	0.016553	0.068518	0.016314	9.35
dbscan_mice_no_context	0.356184	-0.077905	2.344871	0.009983	0.281811	0.282000	8.85
agg_mice_context	0.356184	-0.094494	1.874077	0.004522	0.259546	0.238908	8.45
kmeans_encoded_20	0.289369	-0.212661	28.041057	0.020631	0.190869	0.079326	7.40
agg_encoded_20	0.258243	-0.316824	1.109202	0.043354	0.303899	0.182562	5.75
dbscan_knn_no_context	0.143811	-0.436810	239.623244	0.003395	0.178531	0.012615	4.75
dbscan_knn_context	0.139237	-0.405025	220.061266	0.003395	0.174882	0.007614	4.45

Ending

```
# Find the dataframe with the name 'agg_rf_no_context' in the sorted list
agg_rf_no_context_df = None
for df, name in all_dfs_sorted:
    if name == 'agg_rf_no_context':
        agg_rf_no_context_df = df
        break

# Check if the dataframe was found and display its head
if agg_rf_no_context_df is not None:
    display(agg_rf_no_context_df.head())
else:
    print("DataFrame 'agg_rf_no_context' not found in all_dfs_sorted.")
```

	age	age_group	gender	marital_status	profession_group	workplace	religion	financial_condition	hometown	latitude	...	wind_deg	clouds_sky	weather_main	weather_descriptio
0	17.99	5	1	5	34	0	5	0	18	23.960600	...	320.0	40.0	3	
1	20.00	5	1	4	25	1	5	2	143	24.899220	...	116.0	33.0	1	
2	14.00	5	0	6	26	2	5	2	86	23.832984	...	190.0	75.0	2	
3	35.87	3	0	4	26	2	5	2	17	24.850066	...	186.0	100.0	6	
4	18.00	5	0	5	26	2	5	2	31	23.764386	...	154.0	40.0	3	

5 rows × 30 columns

DEcoding teh encoded dfs

```
non_encoded = pd.read_excel("/content/drive/MyDrive/capstone/datasets/4_non_encoded.xlsx")
rf_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/11_forest_no_context.xlsx")

# mice_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/7_mice_no_context.xlsx")


agg_rf_no_context.head()
```

	age	age_group	gender	marital_status	profession_group	workplace	religion	financial_condition	hometown	latitude	...	wind_deg	clouds_sky	weather_main	weather_descriptio
0	17.99	5	1	5	34	0	5	0	18	23.960600	...	320.0	40.0	3	
1	20.00	5	1	4	25	1	5	2	143	24.899220	...	116.0	33.0	1	
2	14.00	5	0	6	26	2	5	2	86	23.832984	...	190.0	75.0	2	
3	35.87	3	0	4	26	2	5	2	17	24.850066	...	186.0	100.0	6	
4	18.00	5	0	5	26	2	5	2	31	23.764386	...	154.0	40.0	3	

5 rows × 30 columns

```
import json
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder
import os

# 1. Load the saved LabelEncoders JSON
drive_path = '/content/drive/MyDrive/capstone/'
filename = 'label_encoders.json'
full_path = os.path.join(drive_path, filename)

with open(full_path, 'r') as f:
    label_encoders_serializable = json.load(f)

# 2. Decode the encoded DataFrame
rf_no_context_decoded = rf_no_context.copy()

for col, classes in label_encoders_serializable.items():
    if col in rf_no_context_decoded.columns:
        le = LabelEncoder()
        le.classes_ = np.array(classes)  # convert list to numpy array
        rf_no_context_decoded[col] = le.inverse_transform(rf_no_context_decoded[col])

print("Decoding completed!")
```

```
Decoding completed!
```

```
rf_no_context_decoded.head()
```

	age	age_group	gender	marital_status	profession_group	workplace	religion	financial_condition	hometown	latitude	...	wind_speed	wind_deg	clouds_sky	weather_main	weat
0	17.99	teen	male	relationship	worker	rural	muslim	indebted	brahmanbaria	23.960600	...	3.0	320.0	40.0	haze	
1	20.00	teen	male	rejected	stportsman	semi-urban	muslim	middle	sylhet	24.899220	...	3.0	116.0	33.0	clouds	
2	14.00	teen	female	single	student	urban	muslim	middle	manikganj	23.832984	...	3.6	190.0	75.0	drizzle	
3	35.87	middle-aged	female	rejected	student	urban	muslim	middle	bogura	24.850066	...	3.0	186.0	100.0	rain	
4	18.00	teen	female	relationship	student	urban	muslim	middle	dhaka	23.764386	...	3.0	154.0	40.0	haze	

5 rows × 29 columns

```
rf_no_context_merged = rf_no_context_decoded.merge(
    agg_rf_no_context[["agg_cluster"]],
    left_index=True,
    right_index=True,
    how="left"
)

summary = pd.DataFrame({
    "Column": rf_no_context_merged.columns,
    "Data Type": rf_no_context_merged.dtypes.values,
    "Example Values": [
```

```
            rf_no_context_merged[col].dropna().unique()[:3].tolist()
            for col in rf_no_context_merged.columns
        ]
    })

    summary
```

	Column	Data Type	Example Values
0	age	float64	[17.99, 20.0, 14.0]
1	age_group	object	[teen, middle-aged, adult]
2	gender	object	[male, female, third gender]
3	marital_status	object	[relationship, rejected, single]
4	profession_group	object	[worker, stportsman, student]
5	workplace	object	[rural, semi-urban, urban]
6	religion	object	[muslim, hindu, buddhist]
7	financial_condition	object	[indebted, middle, lower]
8	hometown	object	[brahmanbaria, sylhet, manikganj]
9	latitude	float64	[23.9605999, 24.89922, 23.8329836]
10	longitude	float64	[91.1190893, 91.8685271, 89.9666878]
11	reason	object	[harassment, relationship, marital]
12	time	object	[afternoon, night, morning]
13	method	object	[jumping, hanging, overdose]
14	feels_like	float64	[300.65, 299.17, 298.55]
15	temp_min	float64	[299.15, 295.57, 300.65]
16	temp_max	float64	[299.15, 295.57, 300.65]
17	air_pressure	float64	[1008.0, 1012.0, 1004.0]
18	air_humidity	float64	[83.0, 96.0, 78.0]
19	wind_speed	float64	[3.0, 3.6, 3.05]
20	wind_deg	float64	[320.0, 116.0, 190.0]
21	clouds_sky	float64	[40.0, 33.0, 75.0]
22	weather_main	object	[haze, clouds, drizzle]
23	weather_description	object	[haze, scattered clouds, drizzle]
24	suicide_day	float64	[30.0, 11.0, 25.0]
25	suicide_month	float64	[10.0, 4.0, 9.0]
26	suicide_year	float64	[2020.0, 2022.0, 2021.77]
27	suicide_week	float64	[44.0, 15.0, 39.0]
28	suicide_weekday	object	[Friday, Saturday, Thursday]
29	agg_cluster	int64	[0, 1]

```
import pandas as pd

# ----------------------------
```

```
# Step 1: Select your dataset
# ------------------------------
top_df, top_name = rf_no_context_merged, "agg_rf_no_context"
cluster_col = "agg_cluster"

# ------------------------------
# Step 2: Define feature groups
# ------------------------------
num_features = [
    "age", "feels_like", "temp_min", "temp_max",
    "air_pressure", "air_humidity", "wind_speed", "clouds_sky"
]

cat_features = [
    "age_group", "gender", "marital_status", "profession_group",
    "workplace", "religion", "financial_condition", "reason",
    "time", "method", "weather_main", "weather_description",
    "suicide_weekday"
]

# ------------------------------
# Step 3: Numerical summary
# ------------------------------
num_summary = top_df.groupby(cluster_col)[num_features].agg(["mean", "median", "count"])

# ------------------------------
# Step 4: Categorical summary (mode + %)
# ------------------------------
cat_summary = {}
for col in cat_features:
    modes = top_df.groupby(cluster_col)[col].agg(
        lambda x: x.value_counts().index[0] if len(x.value_counts()) > 0 else None
    )
    mode_perc = top_df.groupby(cluster_col)[col].agg(
        lambda x: (x.value_counts().iloc[0] / len(x) * 100) if len(x.value_counts()) > 0 else None
    )
    cat_summary[col] = [f"{modes[c]} ({mode_perc[c]:.1f}%)" for c in modes.index]

cat_summary_df = pd.DataFrame(cat_summary, index=modes.index)

# ------------------------------
# Step 5: Display nicely in Jupyter
# ------------------------------
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 2000)
pd.set_option("display.max_colwidth", None)

print(f"📊 Cluster profiling for: {top_name}")

print("\n🔹 Numerical Feature Summary:")
display(num_summary)

print("\n🔹 Categorical Feature Summary (mode + %):")
display(cat_summary_df)

# ------------------------------
```

```
# Step 6: Save to Excel (for paper)
# ------------------------------
output_file = f"{top_name}_cluster_profiling.xlsx"
with pd.ExcelWriter(output_file) as writer:
    num_summary.to_excel(writer, sheet_name="Numerical_Summary")
    cat_summary_df.to_excel(writer, sheet_name="Categorical_Summary")

print(f"\n✅ Cluster profiling tables saved to '{output_file}'")
```

```
📊 Cluster profiling for: agg_rf_no_context

🔹 Numerical Feature Summary:
```

	age			feels_like			temp_min			temp_max			air_pressure			air_humidity			wind_speed	
	mean	median	count	mean	median	count	mean	median	count	mean	median	count	mean	median	count	mean	median	count	mean	median
agg_cluster																				
0	27.996140	24.0	759	299.893900	300.6500	759	298.946100	299.510	759	298.946100	299.5100	759	1004.773386	1004.000	759	85.187088	87.000	759	3.521950	3.000
1	25.795315	23.0	858	290.455384	290.4434	858	23.776781	25.606	858	31.797987	32.5425	858	1007.013161	1005.901	858	71.705163	70.115	858	7.266621	7.201

```
🔹 Categorical Feature Summary (mode + %):
```

	age_group	gender	marital_status	profession_group	workplace	religion	financial_condition	reason	time	method	weather_main	weather_description	suicide_weekday
agg_cluster													
0	teen (48.5%)	female (54.8%)	married (48.4%)	student (37.0%)	semi-urban (54.5%)	muslim (87.5%)	lower (54.3%)	family issue (23.1%)	night (36.9%)	hanging (73.3%)	clouds (38.1%)	haze (19.6%)	Saturday (17.9%)
1	teen (39.2%)	male (50.8%)	married (47.6%)	student (39.7%)	rural (50.0%)	muslim (85.4%)	lower (57.6%)	mental (26.5%)	night (35.5%)	hanging (74.1%)	partly cloudy (99.9%)	haze (64.7%)	Sunday (16.9%)

```
✅ Cluster profiling tables saved to 'agg_rf_no_context_cluster_profiling.xlsx'
```

```
# Step 2: Compare context vs no-context groups
context_scores = [score_map[name] for _, name in context_dfs if name in score_map]
no_context_scores = [score_map[name] for _, name in no_context_dfs if name in score_map]

print("✅ Average Composite Score (Context):", np.mean(context_scores))
print("✅ Average Composite Score (No-Context):", np.mean(no_context_scores))
```

```
✅ Average Composite Score (Context): 18.97083333333333
✅ Average Composite Score (No-Context): 18.537499999999998
```

```
# Step 3: Compare imputation strategies
def avg_score_for_group(group):
    return np.mean([score_map[name] for _, name in group if name in score_map])

print("RF Imputation Avg Score:", avg_score_for_group(rf_dfs))
print("KNN Imputation Avg Score:", avg_score_for_group(knn_dfs))
print("MICE Imputation Avg Score:", avg_score_for_group(mice_dfs))
print("No Imputation Avg Score:", avg_score_for_group(no_imputed_dfs))
```

```
RF Imputation Avg Score: 23.64375
KNN Imputation Avg Score: 15.3625
MICE Imputation Avg Score: 17.256249999999998
No Imputation Avg Score: 9.737499999999999
```

```
# Step 4: Examine role of balance metric
balance_values = [(name, all_evaluation_results[name]["balance"], score_map[name])
                  for name in all_evaluation_results.keys()]

balance_df = pd.DataFrame(balance_values, columns=["Dataset", "Balance", "Composite"])
balance_df = balance_df.sort_values("Composite", ascending=False)

print("📊 Top vs bottom models by Balance")
display(balance_df.head(10))
display(balance_df.tail(10))
```

📊 Top vs bottom models by Balance

	Dataset	Balance	Composite
7	agg_rf_no_context	0.685461	27.65
21	kmeans_mice_no_context	0.674009	26.95
6	agg_rf_context	0.685461	26.70
31	spc_rf_no_context	0.685461	25.95
14	dbscan_rf_context	0.685461	25.50
29	spc_mice_no_context	0.673571	25.45
27	spc_knn_no_context	0.538092	24.80
30	spc_rf_context	0.685461	24.80
26	spc_knn_context	0.538092	24.70
23	kmeans_rf_no_context	0.685023	23.65
	Dataset	**Balance**	**Composite**
5	agg_mice_no_context	0.356184	9.85
2	agg_knn_context	0.358801	9.65
17	kmeans_encoded_40	0.315207	9.35
3	agg_knn_no_context	0.360987	9.35
13	dbscan_mice_no_context	0.356184	8.85
4	agg_mice_context	0.356184	8.45
16	kmeans_encoded_20	0.289369	7.40
0	agg_encoded_20	0.258243	5.75
11	dbscan_knn_no_context	0.143811	4.75
10	dbscan_knn_context	0.139237	4.45

```
# Step 5: Best model per clustering algorithm
best_per_algo = {}
for algo, group in [("Agglomerative", agg_dfs),
                    ("KMeans", kmeans_dfs),
                    ("DBSCAN", dbscan_dfs),
                    ("Spectral", spc_dfs)]:
    scores = [(name, score_map[name]) for _, name in group if name in score_map]
    best_per_algo[algo] = max(scores, key=lambda x: x[1])

print("🏆 Best model per clustering algorithm:")
for algo, (name, score) in best_per_algo.items():
    print(f"{algo}: {name} → Score {score:.2f}")
```

```
🏆 Best model per clustering algorithm:
Agglomerative: agg_rf_no_context → Score 27.65
KMeans: kmeans_mice_no_context → Score 26.95
```

```
DBSCAN: dbscan_rf_context → Score 25.50
Spectral: spc_rf_no_context → Score 25.95
```

```
# Step 6: Extended social profiling for clusters

categorical_vars = [
    "gender", "marital_status", "profession_group",
    "financial_condition", "reason", "method",
    "workplace", "age_group"
]

for col in categorical_vars:
    print(f"\n📊 Distribution of {col} across clusters in {top_name}")
    dist_table = top_df.groupby(cluster_col)[col].value_counts(normalize=True).unstack(fill_value=0) * 100
    dist_table = dist_table.round(1)  # round to 1 decimal for clarity
    display(dist_table)
```

📊 Distribution of gender across clusters in agg_rf_no_context

agg_cluster	gender female	male	third gender
0	54.8	45.1	0.1
1	48.7	50.8	0.5

📊 Distribution of marital_status across clusters in agg_rf_no_context

agg_cluster	marital_status broken up	divorced	engaged	married	rejected	relationship	single	widowed
0	1.1	0.7	0.4	48.4	11.2	9.0	29.1	0.3
1	0.0	2.2	0.0	47.6	2.3	4.2	43.7	0.0

📊 Distribution of profession_group across clusters in agg_rf_no_context

agg_cluster	profession_group banker	building manager	business	criminal	cultural worker	engineer	entertainment	expatriate	farmer	fisherman	freelancer	healthcare	hospitality	housekeeper	housewife
0	0.0	0.0	1.6	0.7	0.0	0.1	0.1	0.0	1.2	0.0	0.0	0.4	0.1	0.0	20.8
1	0.1	0.1	3.5	0.8	0.1	0.0	0.3	1.6	1.0	0.3	0.1	1.4	0.1	0.3	17.0

📊 Distribution of financial_condition across clusters in agg_rf_no_context

agg_cluster	financial_condition indebted	lower	middle	upper	upper_middle
0	2.6	54.3	39.9	1.2	2.0
1	4.7	57.6	32.2	3.6	2.0

📊 Distribution of reason across clusters in agg_rf_no_context

agg_cluster	reason abandonment and despair	abuse	accident	addiction	child marriage	crime	disagreement over marriage	divorce	due to pride	economic crisis	education	extramarital affair	family	family issue	financial	grief over wife's sudden death	harassment	s
0	0.0	0.0	0.0	0.8	0.0	0.0	0.0	0.1	0.0	0.0	5.0	0.0	0.0	23.1	2.9	0.8	4.3	
1	0.1	0.1	0.9	0.6	0.1	0.8	0.1	0.0	0.1	0.1	1.0	0.2	0.6	25.8	2.7	0.1	2.1	

📊 Distribution of method across clusters in agg_rf_no_context

agg_cluster	method cutting	drowning	electrocution	hanging	injection	jumping	overdose	self-immolation	shooting	suffocation	suicide	train accident
0	0.8	0.0	0.1	73.3	2.1	2.6	20.0	0.0	0.4	0.0	0.0	0.7
1	1.5	0.6	0.1	74.1	1.7	4.1	14.7	1.2	1.6	0.1	0.1	0.1

📊 Distribution of workplace across clusters in agg_rf_no_context

agg_cluster	workplace	rural	semi-urban	urban
0		35.7	54.5	9.7
1		50.0	23.4	26.6

📊 Distribution of age_group across clusters in agg_rf_no_context

agg_cluster	age_group	adult	baby	child	middle-aged	old	teen	young
0		9.2	0.1	0.7	20.7	4.1	48.5	16.7
1		28.3	0.7	1.2	3.6	2.7	39.2	24.4

```
# Step 5: Best model per clustering algorithm
best_per_algo = {}
for algo, group in [("Agglomerative", agg_dfs),
                    ("KMeans", kmeans_dfs),
                    ("DBSCAN", dbscan_dfs),
                    ("Spectral", spc_dfs)]:
    scores = [(name, score_map[name]) for _, name in group if name in score_map]
    best_per_algo[algo] = max(scores, key=lambda x: x[1])

print("🏆 Best model per clustering algorithm:")
for algo, (name, score) in best_per_algo.items():
    print(f"{algo}: {name} → Score {score:.2f}")
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipython-input-2026774172.py in <cell line: 0>()
      1 # Step 5: Best model per clustering algorithm
      2 best_per_algo = {}
----> 3 for algo, group in [("Agglomerative", agg_dfs),
      4                     ("KMeans", kmeans_dfs),
      5                     ("DBSCAN", dbscan_dfs),

NameError: name 'agg_dfs' is not defined
```